

Secure Market

A Secure Implementation Report for a security-focused marketplace platform,
Part of the Course for the Discipline of DESOFS
of the Masters program in Cybersecurity and System's Administration

by

Pedro D Nunes*

Pedro Leal[†]

Mário Baptista[‡]

Diogo Teixeira[§]

Supervised by Professor Cláudio Maia

Submitted April 20, 2026

*Student ID: 1250543, Email: 1250543@isep.ipp.pt

[†]Student ID: 1211066, Email: 1211066@isep.ipp.pt

[‡]Student ID: 1211265, Email: 1211265@isep.ipp.pt

[§]Student ID: 1070370, Email: 1070370@isep.ipp.pt

Table of Contents

<i>List of Figures</i>	6
<i>List of Tables</i>	7
<i>Glossary</i>	8
1 Project Overview	10
2 System Requirements	11
2.1 Functional Requirements	11
2.1.1 FR-01 - Sellers should be allowed to execute CRUD operations with products	11
2.1.2 FR-02 - Sellers should be allowed to confirm pending orders	12
2.1.3 FR-03 - Buyers should be allowed to place orders	12
2.1.4 FR-04 - Buyers should be able to download invoices	12
2.1.5 FR-05 - Buyers should be allowed to review purchased products	13
2.1.6 FR-06 - System should generate invoices after a successful purchase . .	13
2.1.7 FR-07 - Admins should be able to inactivate users that break SecureMarket rules	13
2.1.8 FR-08 - Admins should be able to remove spam/offensive reviews . . .	13
2.1.9 FR-09 - Admins should be able to download invoices	14
2.1.10 FR-10 - Admins should be able to set other users as Admins	14
2.1.11 FR-AC-01 - Guests should be able to register using email and password	14
2.1.12 FR-AC-02 - Users should be able to login using valid credentials	14
2.1.13 FR-AC-03 - Users should be able to logout	15
2.1.14 FR-AC-04 - Users should be authenticated before having access to protected resources	15

2.1.15	FR-AC-05 - System should restrict access to system functionalities based on user roles	15
2.1.16	FR-AC-06 - Users should be able to reset their password	16
2.2	Non Functional Requirements	16
2.2.1	NFR-01 - System should respond to non-file API requests within 500ms	16
2.2.2	NFR-02 - System should enforce HTTPS for all client-server communication	16
2.2.3	NFR-03 - System should store sensitive data using hashing algorithms .	16
2.2.4	NFR-04 - System should have rate-limits to avoid brute-force attacks .	17
2.2.5	NFR-05 - System should end sessions after an inactivity period	17
2.2.6	NFR-06 - System should validate the size and content of uploaded files	17
2.2.7	NFR-07 - System should validate the input data	17
2.2.8	NFR-08 - System should log critical operations	17
2.2.9	NFR-09 - System should not disclose sensitive or application information on error messages	17
2.3	Abuse Cases	18
3	Threat Modelling	20
3.1	Methodology	20
3.1.1	Threat Identification — STRIDE	20
3.1.2	Risk Assessment — DREAD	20
3.2	STRIDE per DFD Element	21
3.2.1	External Entities	21
3.2.1.1	EE-1: Admin / Seller / Buyer (All External Users)	21
3.2.2	Process Threats	22
3.2.2.1	P-1: Authentication & Session Management	22
3.2.2.2	P-2: Authorisation & Access Control	23
3.2.2.3	P-3: Product Management	24

3.2.2.4	P-4: Order Management	24
3.2.2.5	P-5: Image Management	25
3.2.2.6	P-6: Invoice Generation	26
3.2.2.7	P-7: Review Management	26
3.2.2.8	P-8: Export Generation	27
3.2.2.9	P-9: Audit Logging	27
3.3	Data Flow Threats	28
3.4	Risk Assessment Summary	29
3.5	Mitigation Traceability Matrix	30
4	Architecture and Data Flows	33
4.1	Architecture Overview	33
4.2	Architecture Domain Model	35
4.2.0.1	User Aggregate	35
4.2.0.2	Product Aggregate	35
4.2.0.3	Order Aggregate	35
4.3	Data Flow Diagrams	36
4.3.1	DFD Level 0 – System Context	36
4.3.2	DFD Level 1 – Process Decomposition	37
4.3.3	DFD Level 2 – Authentication & Session Management	38
4.3.4	DFD Level 2 – Order Management	38
4.3.5	DFD Level 2 – Image Management	39
4.3.6	DFD Level 2 – Invoice Generation	41
5	Secure Development Requirements	43
5.1	Authentication & Session Management (SR-AUTH)	43
5.2	Authorization & Access Control (SR-AUTHZ)	45

5.3	Data Security & Integrity (SR-DATA)	46
5.4	Communication Security (SR-COMM)	48
5.5	Input Validation & Data Handling (SR-INPUT)	49
5.6	Third-Party Components (SR-3RD)	51
5.7	Logging & Monitoring (SR-LOG)	52
5.8	Business Logic Security (SR-BIZ)	53
5.9	Requirements Traceability Matrix	54
6	Secure Design	55
6.1	Secure Design Principles	55
6.2	Role-Based Access Control Design	56
7	Secure Architecture	57
7.1	Layered Architecture Overview	57
7.2	Deployment Architecture	58
7.3	Security Middleware Stack	59
7.4	Authentication Architecture	59
7.5	Database Security Architecture	60
7.6	CI/CD Pipeline Security Architecture	61
8	Security Testing Plan	62
8.1	Testing Methodology	62
8.2	Quality Gates	62
8.3	Static Analysis (SAST) Test Cases	63
8.4	Software Composition Analysis (SCA) Test Cases	64
8.5	Dynamic Analysis (DAST) Test Cases	65
8.6	Manual Security Testing	66
8.6.1	Authentication Abuse Tests	66

8.6.2	Authorisation Abuse Tests	66
8.6.3	Business Logic Abuse Tests	67
8.6.4	File Operation Abuse Tests	69
8.7	Test Traceability Matrix	69

<i>Bibliography</i>	71
----------------------------	-----------

List of Figures

1	AC-02 diagram	19
2	AC-05 diagram	19
3	Architecture Overview Diagram	34
4	Domain Model Diagram	36
5	DFD Level 0	37
6	DFD Level 1	37
7	DFD Level 2 – Authentication & Session Management	38
8	DFD Level 2 – Order Management	39
9	DFD Level 2 – Image Management	41
10	DFD Level 2 – Invoice Generation	42
11	SecureMarket Four-Layer Architecture	57

List of Tables

1	Functional Requirements	11
2	Authentication and Access Control - Functional Requirements	11
3	Non Functional Requirements	16
4	Abuse Cases	18
5	STRIDE Threat Categories	20
6	DREAD Risk Scoring Factors	21
7	Threats on External Entities	22
8	Threats on Authentication & Session Management	22
9	Threats on Authorisation & Access Control	23
10	Threats on Product Management	24
11	Threats on Order Management	24
12	Threats on Image Management	25
13	Threats on Invoice Generation	26
14	Threats on Review Management	27
15	Threats on Export Generation	27
16	Threats on Audit Logging	28
17	Data Flow Threats	28
18	DREAD Risk Scores per Threat	29
19	Threats to Mitigations Mapping	30
29	Secure Design Principles	55
30	Role-Based Access Control Permissions Matrix	56
31	Layer Responsibilities	57
32	Deployment Components and Security Configuration	58
33	Security Middleware Stack	59
34	Authentication Architecture	60
35	Database Security Architecture	60
36	CI/CD Pipeline Security Stages and Verification Controls	61
37	Security Testing Methodology	62
38	Security Quality Gates	62
39	Static Analysis Security Test Cases	63
40	Software Composition Analysis Test Cases	64
41	Dynamic Analysis Security Test Cases	65
42	Authentication Abuse Test Cases	66
43	Authorisation Abuse Test Cases	67
44	Business Logic Abuse Test Cases	68
45	File Operation Abuse Test Cases	69
46	Security Test Traceability Matrix	70

Glossary

ASVS - An acronym for **A**pplication **S**ecurity **V**erification **S**tandard. A project by OWASP that provides a basis for testing web application technical security controls and also provides developers with a list of requirements for secure development.

DAST - **D**ynamic **A**pplication **S**ecurity **T**esting. A testing process to identify security weaknesses and vulnerabilities in an application. Can be manual or use automated tools.

DFD - **D**ata **F**low **D**iagram. A way of representing a flow of data through a process or a system.

DREAD - A risk assessment model used to prioritize threats based on five factors: **D**amage **R**eproducibility **E**xploitability **A**ffected Users **D**iscoverability.

IDOR — Insecure Direct Object Reference. A type of access control vulnerability where an attacker manipulates a resource identifier (such as a UUID in a URL) to gain unauthorised access to another user's data.

JWT — JSON Web Token. A compact, URL-safe token format used to represent authentication claims between two parties. A JWT consists of a Base64-encoded header, payload, and cryptographic signature separated by dots.

MITM — Man-in-the-Middle. An attack where an adversary secretly intercepts and potentially alters communications between two parties who believe they are communicating directly with each other.

OWASP - An acronym for **O**pen **W**orldwide **A**pplication **S**ecurity **P**roject. A non-profit organization that provides open papers and resources to improve software security, by defining standards and publishing guidelines, tools and best practices for secure development, such as OWASP Top 10.

SAST - An acronym for **S**tatic **A**pplication **S**ecurity **T**esting. Source code analysis to detect vulnerabilities.

SCA - An acronym for **S**oftware **C**omposition **A**nalysis. Identifies vulnerabilities in third-party libraries and dependencies.

SSDLC - An acronym for **S**ecure **S**oftware **D**evelopment **L**ife **C**ycle. An extension of Software Development Life Cycle that adds security practices on every software development phase.

STRIDE - A threat modeling methodology used to identify security threats based on six categories: **S**poofing **T**ampering **R**epudiation **I**nformation disclosure **D**enial of Service **E**levation of privilege.

UUID - **U**niversally **U**nique **I**Dentifier. A 128-bit number used to identify information in computer systems.

XSS - Cross-Site Scripting. A class of injection attack where malicious scripts are injected into content that is later rendered in a victim's browser or application, allowing the attacker to execute code in the victim's context.

FastAPI - A modern, high-performance web framework for building APIs with Python 3.7+ based on standard Python type hints, used as the foundation for the SecureMarket platform.

HSTS - **H**TTP **S**trict **T**ransport **S**ecurity. A security mechanism that forces the use of HTTPS connections, preventing downgrade attacks and cookie hijacking.

Jinja2 - A templating engine for Python. It is used in SecureMarket for generating invoice files, requiring strict sandboxing to prevent template injection attacks.

ORM - Object-Relational Mapping. A programming technique for converting data between incompatible type systems using object-oriented programming languages. It is used to mitigate SQL injection risks.

PostgreSQL - A open-source object-relational database system used to store the structured data for the SecureMarket platform.

RBAC - **R**ole-**B**ased **A**ccess **C**ontrol. A method of restricting system access to authorized users based on their assigned roles within the system.

Reverse Proxy - A server that sits in front of web servers and forwards client requests to those web servers. It is responsible for enforcing TLS and forwarding traffic to the application.

Template Injection - A vulnerability where an attacker can inject a malicious payload into a template engine (such as Jinja2), potentially leading to remote code execution on the server.

TLS - **T**ransport **L**ayer **S**ecurity. A cryptographic protocol designed to provide communications security over a computer network, enforced for all client-server communication in the system.

1 Project Overview

SecureMarket is a security-focused marketplace platform designed to ensure sellers, buyers, and administrators can perform their actions safely.

The system was designed as a secure REST API. It allows sellers to manage their product listings with images, enables buyers to purchase sellers' products and manage their orders, and allows administrators to manage the platform.

The platform was designed using a layered FastAPI implementation with a Domain-Driven Design approach. These design decisions were taken to include security controls such as server-side price integrity, role-based access control, filesystem-based image and invoice handling with UUID-based filenames, and comprehensive audit logging. The project was threat-modeled (following STRIDE and DREAD), data-flow analyzed, and security requirements were designed following OWASP ASVS in order to ensure that every feature was designed with security in mind.

2 System Requirements

This section presents the System Requirements, divided into three categories: Functional Requirements, Non Functional Requirements and Abuse Cases.

2.1 Functional Requirements

We have divided our functional requirements into two categories: general requirements related to the market portal business rules (10 requirements) and related to the Authentication/Access Control (6 requirements).

Table 1: Functional Requirements

ID	Functional Requirement
FR-01	Sellers should be allowed to execute CRUD operations with products.
FR-02	Sellers should be allowed to confirm pending orders.
FR-03	Buyers should be allowed to place orders.
FR-04	Buyers should be able to download invoices.
FR-05	Buyers should be allowed to review purchased products.
FR-06	System should generate invoices after a successful purchase.
FR-07	Admins should be able to inactivate users that break SecureMarket rules.
FR-08	Admins should be able to remove spam/offensive reviews.
FR-09	Admins should be able to download invoices.
FR-10	Admins should be able to set other users as Admins.

Table 2: Authentication and Access Control - Functional Requirements

ID	Functional Requirement
FR-AC-01	Guests should be able to register using email and password.
FR-AC-02	Users should be able to login using valid credentials.
FR-AC-03	Users should be able to logout.
FR-AC-04	Users should be authenticated before having access to protected resources.
FR-AC-05	System should restrict access to system functionalities based on user roles.
FR-AC-06	Users should be able to reset their password.

2.1.1 FR-01 - Sellers should be allowed to execute CRUD operations with products

Actors - Users with SELLER Profile

Description - The system should allow users with SELLER profile to create, update, read and delete products. Among the product data, users can change the product state to Draft, Suspended, Archived and Active.

Preconditions - User is authenticated

Postconditions - If the user input is valid, operation should succeed

Security Notes - Validate user input

2.1.2 FR-02 - Sellers should be allowed to confirm pending orders

Actors - Users with SELLER Profile

Description - The system should allow users with SELLER profile to update pending orders to the following states: Shipped or Cancelled.

Preconditions - User is authenticated

Postconditions - Order is updated with the state defined by the seller.

Security Notes - Validate if new state is valid.

2.1.3 FR-03 - Buyers should be allowed to place orders

Actors - Users with BUYER Profile

Description - The system should allow users with BUYER profile to buy a product from a seller.

Preconditions - User is authenticated and there are products on Active state.

Postconditions - Order is created successfully on pending state.

Security Notes - Validate user input to ensure that price or product characteristics were not manipulated

2.1.4 FR-04 - Buyers should be able to download invoices

Actors - Users with BUYER Profile

Description - The system should allow users with BUYER profile to download invoices for the purchases they made.

Preconditions - User is authenticated and has previous purchases

Postconditions - Invoice file is downloaded.

Security Notes - Validate user input to ensure that the requested purchase was made by the requesting user.

2.1.5 FR-05 - Buyers should be allowed to review purchased products

Actors - Users with BUYER Profile

Description - The system should allow users with BUYER profile to review purchased products.

Preconditions - User is authenticated and has previous purchases in state Delivered

Postconditions - Review is added and visible to all users when viewing the product.

Security Notes - Validate user input for malicious data. Ensure that the product was purchased by the user.

2.1.6 FR-06 - System should generate invoices after a successful purchase

Actors - Users with BUYER Profile

Description - The system should automatically generate the invoice after a successful purchase.

Preconditions - A purchase was successfully created.

Postconditions - Invoice file is generated and stored.

Security Notes - Generate invoices only for successfully created purchases.

2.1.7 FR-07 - Admins should be able to inactivate users that break SecureMarket rules

Actors - Users with ADMIN Profile

Description - The system should allow users with ADMIN profile to inactivate any user. Should also be possible to reactivate inactivated users.

Preconditions - User exists and is on Active state

Postconditions - User is set as inactive. The inactivated user should not be able to make any operation other than login and logout.

Security Notes - The user logged in should not be able to inactivate himself.

2.1.8 FR-08 - Admins should be able to remove spam/offensive reviews

Actors - Users with ADMIN Profile

Description - The system should allow users with ADMIN profile to remove any product review.

Preconditions - Product review must exist

Postconditions - Product review is removed.

Security Notes - Validate user input to ensure that the review to be removed exists.

2.1.9 FR-09 - Admins should be able to download invoices

Actors - Users with ADMIN Profile

Description - The system should allow users with ADMIN profile to download any invoice.

Preconditions - User is authenticated and purchases have been made by any user

Postconditions - Invoice file is downloaded.

Security Notes - Validate user input to ensure that the requested invoice exists.

2.1.10 FR-10 - Admins should be able to set other users as Admins

Actors - Users with ADMIN Profile

Description - The system should allow users with ADMIN profile to set other user as Administrator.

Preconditions - User is authenticated and there are additional users that do not have ADMIN profile

Postconditions - The edited user has ADMIN profile, along with other previous profiles he had.

Security Notes - Validate user input to ensure that the user to promote as administrator exists and is not an administrator already.

2.1.11 FR-AC-01 - Guests should be able to register using email and password

Actors - Guests

Description - The system should allow guests to register using a email and password.

Preconditions - There is no active session for a registered user

Postconditions - A new user account is created.

Security Notes - Prevent duplicate accounts, enforce password policy and ensure there is no malicious input.

2.1.12 FR-AC-02 - Users should be able to login using valid credentials

Actors - Registered, unauthenticated users

Description - The system should allow unauthenticated users that present valid credentials to login.

Preconditions - There is an active user account created for the given username and user is not authenticated.

Postconditions - User is logged in successfully.

Security Notes - Protect against brute force attacks. Validate user input to avoid malicious data.

2.1.13 FR-AC-03 - Users should be able to logout

Actors - Authenticated users

Description - Users should be able to end their session.

Preconditions - User is authenticated

Postconditions - User session is terminated.

Security Notes - Ensure session is terminated and no further access to resources can be made.

2.1.14 FR-AC-04 - Users should be authenticated before having access to protected resources

Actors - System

Description - The system should validate if user is authenticated before granting access to protected resources.

Preconditions - A request to access a protected resource is made

Postconditions - Access to the resource is granted if the user is valid, denied otherwise.

Security Notes - Ensure security checks are valid and up to date.

2.1.15 FR-AC-05 - System should restrict access to system functionalities based on user roles

Actors - Authenticated users

Description - The system should restrict access to system functionalities based on user roles (ADMIN, SELLER, BUYER).

Preconditions - User is authenticated and has at least one role assigned

Postconditions - Access is granted or denied based on the roles the user has.

Security Notes - Validate if user is not trying to grant himself privileges he does not have.

2.1.16 FR-AC-06 - Users should be able to reset their password

Actors - Registered user

Description - The system should allow users to reset their password.

Preconditions - User account exists

Postconditions - Password is updated successfully.

Security Notes - Validate user input to enforce password policies. Ensure that reset tokens are valid and have not been used yet.

2.2 Non Functional Requirements

We have identified the following non-functional requirements on our SecureMarket project:

Table 3: Non Functional Requirements

ID	Non Functional Requirement
NFR-01	System should respond to non-file API requests within 500ms.
NFR-02	System should enforce HTTPS for all client-server communication.
NFR-03	System should store sensitive data using hashing algorithms.
NFR-04	System should have rate-limits to avoid brute-force attacks.
NFR-05	System should end sessions after an inactivity period.
NFR-06	System should validate the size and content of uploaded files.
NFR-07	System should validate the input data.
NFR-08	System should log critical operations.
NFR-09	System should not disclose sensitive or application information on error messages.

2.2.1 NFR-01 - System should respond to non-file API requests within 500ms

Under normal load conditions, all requests to the API should have a response time under 500ms, with the exception of file download/upload operations,

2.2.2 NFR-02 - System should enforce HTTPS for all client-server communication

All traffic between client applications and the SecureMarket API must be made using HTTPS.

2.2.3 NFR-03 - System should store sensitive data using hashing algorithms

Sensitive data, such as passwords must not be stored in plain text. A hashed value should be generated and stored instead.

2.2.4 NFR-04 - System should have rate-limits to avoid brute-force attacks

To avoid brute-force[1] and DDoS[2] attacks, the following rate limits should be implemented:

- For authentication endpoints, a maximum of 5 requests per minute and IP. This limit is expected to avoid brute-force attacks.
- For other endpoints, a maximum of 100 requests/minute. This limit is expected to avoid DDoS attacks.

If the limits are exceeded, requests from the IP should be blocked for a period of 15 minutes. Additionally, for login attempts, after 5 consecutive failed attempts the user should be locked for 30 minutes.

2.2.5 NFR-05 - System should end sessions after an inactivity period

After a user is inactive for more than 30 minutes, its session should be automatically terminated.

2.2.6 NFR-06 - System should validate the size and content of uploaded files

Users should not be able to attach files over 10MB. In addition, the file should be validated to check if the content matches the extension.

2.2.7 NFR-07 - System should validate the input data

Input data received on requests should be validated to ensure that:

- User does not input data to try to interfere with the system database (SQL Injection)
- User does not input data to try to gain advantage and override the system business rules (Data Tampering).

2.2.8 NFR-08 - System should log critical operations

Critical Operations, such as the ones related to authentication/authorization, should be logged in a structured way so that they can be easily monitored.

2.2.9 NFR-09 - System should not disclose sensitive or application information on error messages

When a request made by a user fails, the error message returned by the application should be easy to understand and do not include information that is not essential for the user to solve the problem.

Scenarios to consider:

- Do not include data that may contribute to brute-force attacks. E.g. when login fails due to invalid credentials, the message should be the same if the user exists or not.

- Do not include information that describes details on the SecureMarket implementation, such as database or table names, stack traces, among others.
- Ensure that the HTTPS requests status code is adequate to the error that occurred.

2.3 Abuse Cases

To define the abuse cases for the SecureMarket application, we have analysed the Use Cases and identified what actions a malicious user might execute to take advantage. We have identified eight abuse cases, distributed by various areas of the application (business rules, access control, service disruption). Some of the abuse cases were represented with a diagram.

Table 4: Abuse Cases

Abuse case	Description	Actor	Func. Req	Mitigation
AC-01	Seller uploads a malicious file on a product	User	FR-01	NFR-06
AC-02	Buyer manipulates the input data when creating a new order, reducing the product price	User	FR-03	NFR-07
AC-03	Assign ADMIN role without having permissions to do so	User	FR-10	FR-AC-05 NFR-08
AC-04	Buyer adds product reviews with spam or malicious data	User	FR-05	FR-08
AC-05	Guest makes multiple attempts to login (Brute-force)	Guest	FR-AC-02	NFR-04 NFR-08 NFR-09
AC-06	User tries to download a invoice he's not authorized to	User	FR-04	FR-AC-05 NFR-08
AC-07	User makes a abnormal number of API requests on a short period of time (DDoS)	User	NFR-01	NFR-04 NFR-08
AC-08	Guests create fake user accounts	Guest	FR-AC-01	FR-07 NFR-04 NFR-08

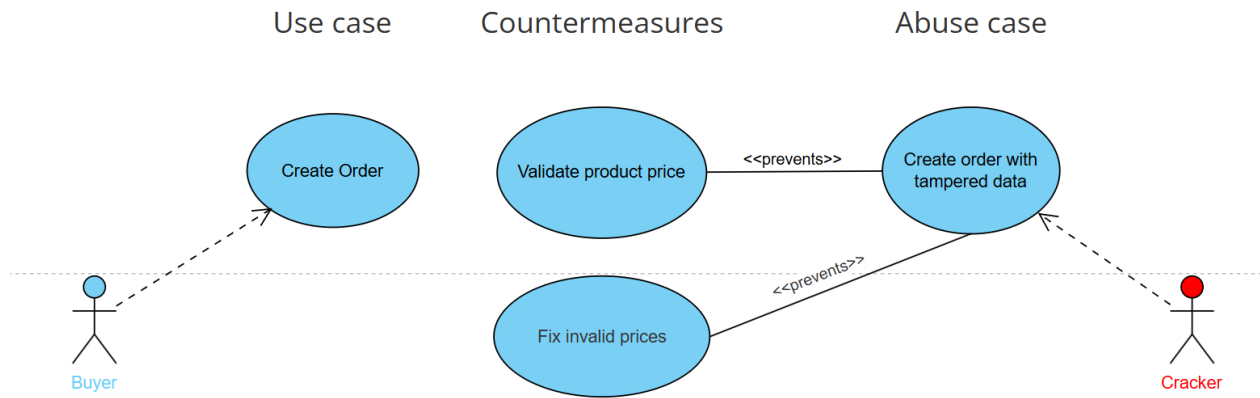


Figure 1: AC-02 diagram

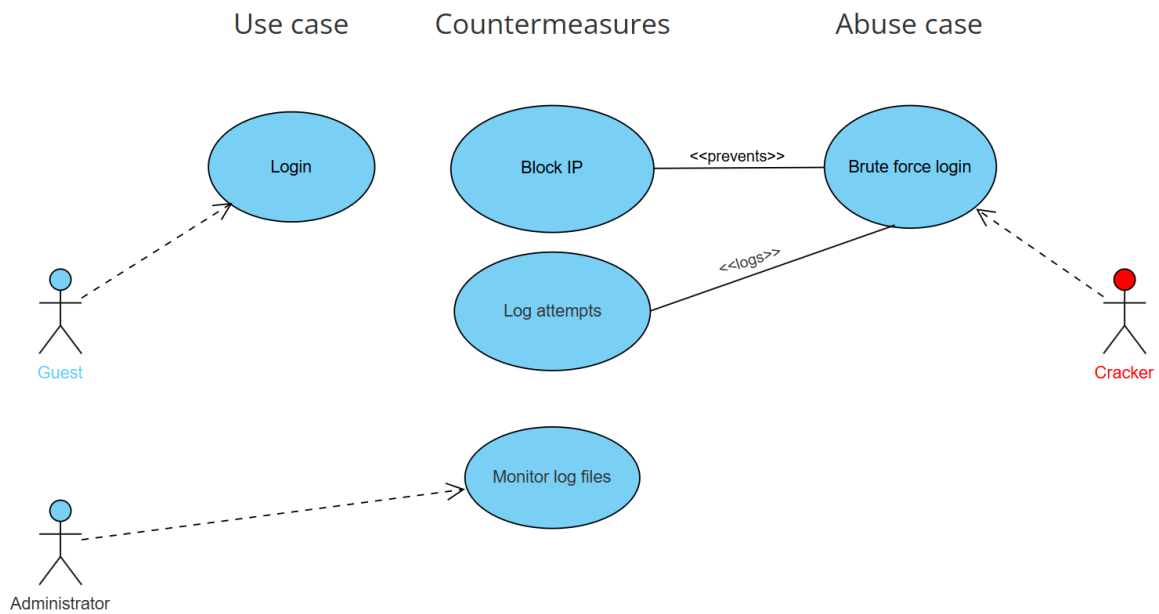


Figure 2: AC-05 diagram

3 Threat Modelling

This section presents the threat analysis performed on the SecureMarket platform. Threats were identified using the STRIDE [3] methodology applied per DFD element, and each threat was assessed using the DREAD [4] risk model to determine its priority. The output of this analysis directly informed the security requirements defined in Section Secure Development Requirements.

3.1 Methodology

3.1.1 Threat Identification — STRIDE

Threats are identified by applying **STRIDE per Element** to each component of the Level 1 DFD. STRIDE is a structured threat categorisation model, which enumerates six threat categories and maps each to the security property it violates.

Table 5: STRIDE Threat Categories

Category	Code	Description	Violated Property
Spoofing	S	Impersonating a user or system	Authentication
Tampering	T	Unauthorised modification of data	Integrity
Repudiation	R	Denying having performed an action	Non-repudiation
Info. Disclosure	I	Exposing data to unauthorised parties	Confidentiality
Denial of Service	D	Making the system unavailable	Availability
Elev. of Privilege	E	Gaining unauthorised access level	Authorisation

3.1.2 Risk Assessment — DREAD

Each identified threat is scored using the **DREAD** model, where each of the five factors is rated from 1 (low) to 3 (high). The overall risk score is the average of all five factors.

Table 6: DREAD Risk Scoring Factors

Factor	Description	1 — Low	2 — Medium	3 — High
Damage	Impact if exploited	Minor inconvenience	Significant data/financial loss	Full system compromise
Reproducibility	Ease of reproducing	Difficult, rare	Moderate setup needed	Trivial, always works
Exploitability	Skill level required	Expert, custom tools	Intermediate	Beginner, automated tools
Affected Users	Scope of impact	Single user	Subset of users	All users
Discoverability	Ease of finding	Hidden, obscure	Requires investigation	Obvious, public knowledge

The risk score is computed as:

$$\text{Risk Score} = \frac{D + R + E + A + D}{5}$$

Scores map to the following priority levels:

- **Critical** Score 2.5–3.0
- **High** Score 2.0–2.4
- **Medium** Score 1.5–1.9
- **Low** Score 1.0–1.4

3.2 STRIDE per DFD Element

3.2.1 External Entities

3.2.1.1 EE-1: Admin / Seller / Buyer (All External Users) External users are the human actors that interact with SecureMarket through the public API. They represent the primary source of untrusted input and are the most common target and origin of spoofing and repudiation threats.

Table 7: Threats on External Entities

ID	Cat.	Threat	Attack Vector
T-EE-01	S	Attacker impersonates a legitimate user by stealing or guessing credentials.	Credential stuffing, brute force, phishing, session hijacking.
T-EE-02	S	Attacker uses a stolen or leaked JWT token to impersonate an authenticated user.	Token theft via XSS, network sniffing on non-TLS connections, or token inadvertently recorded in server logs.
T-EE-03	R	A user performs a malicious action (e.g., places a fraudulent order or deletes products) and later denies responsibility.	Absence of an immutable audit trail or use of shared accounts.

3.2.2 Process Threats

3.2.2.1 P-1: Authentication & Session Management The authentication process is the first line of defence against unauthorised access. Weaknesses here allow attackers to bypass all downstream controls.

Table 8: Threats on Authentication & Session Management

ID	Cat.	Threat	Attack Vector
T-P1-01	S	Brute force attack against the login endpoint to guess user credentials.	Automated tools submitting large volumes of password guesses.
T-P1-02	S	Credential stuffing using known breached credential databases.	Automated replay of email/password pairs from third-party data breaches.
T-P1-03	T	Attacker modifies the JWT token payload to alter the role or user ID claim.	Crafting a JWT with altered claims if the signing secret is weak or leaked.

(continued)

ID	Cat.	Threat	Attack Vector
T-P1-04	I	JWT signing secret exposed through source code, logs, or configuration leaks.	Hardcoded secrets in source code or <code>.env</code> files committed to version control.
T-P1-05	D	Account lockout abuse — attacker deliberately triggers lockout for legitimate users.	Targeted lockout of seller accounts during peak sales periods.
T-P1-06	E	Attacker exploits a weak token refresh mechanism to maintain access after privilege revocation.	Using a valid refresh token after the user's account has been deactivated.

3.2.2.2 P-2: Authorisation & Access Control Authorisation threats arise when the system fails to enforce boundaries between roles or between resources owned by different users.

Table 9: Threats on Authorisation & Access Control

ID	Cat.	Threat	Attack Vector
T-P2-01	E	IDOR — accessing another user's resources by manipulating IDs in API URLs.	Changing <code>/orders/{id}</code> to another buyer's order UUID.
T-P2-02	E	Missing function-level access control — a buyer accesses admin-only endpoints.	Directly calling <code>/api/v1/users</code> or <code>/api/v1/audit-log</code> without admin privileges.
T-P2-03	E	Privilege escalation via parameter tampering — a user modifies their own role through an API payload.	Including <code>"role": "ADMIN"</code> in a profile update request body.
T-P2-04	T	Resource-level bypass — a seller modifies another seller's product.	Manipulating the <code>product_id</code> in an update request.

3.2.2.3 P-3: Product Management Product management threats target data integrity of listings and the trust buyers place in the information shown to them.

Table 10: Threats on Product Management

ID	Cat.	Threat	Attack Vector
T-P3-01	T	Seller manipulates a product price after a buyer has placed an order.	Changing the price between order placement and confirmation to overcharge the buyer.
T-P3-02	T	Stored XSS injected in product title, description, or review content.	Injecting malicious script tags in text fields rendered in reports or a future frontend.
T-P3-03	I	Unauthorised access to DRAFT or SUSPENDED product details by buyers.	IDOR or missing status-based filtering on the product endpoints.
T-P3-04	T	Stock manipulation — negative stock arises from race conditions on concurrent orders.	Multiple concurrent order confirmations decrementing stock below zero.

3.2.2.4 P-4: Order Management Order management threats are among the highest-impact in the system, as they directly affect financial integrity and the correctness of the purchase lifecycle.

Table 11: Threats on Order Management

ID	Cat.	Threat	Attack Vector
T-P4-01	T	Price manipulation at order time — buyer submits a crafted request with artificially low prices.	API accepts client-supplied prices instead of performing a server-side lookup.
T-P4-02	I	Buyer accesses another buyer’s order details or invoice.	IDOR on order endpoints using a guessed or enumerated order UUID.

(continued)

ID	Cat.	Threat	Attack Vector
T-P4-03	T	Order status manipulation — buyer forces a status transition to DELIVERED in order to leave fraudulent reviews.	Bypassing the enforced order state machine.
T-P4-04	R	Seller claims they never received an order or shipped items.	Insufficient lifecycle logging for order events.
T-P4-05	T	Race condition in order placement — multiple simultaneous orders are accepted for the last unit in stock.	Concurrent requests bypassing the stock availability check.

3.2.2.5 P-5: Image Management Image management introduces filesystem attack surface. Threats in this category can affect server integrity and data confidentiality if not properly controlled.

Table 12: Threats on Image Management

ID	Cat.	Threat	Attack Vector
T-P5-01	T	Malicious file upload — attacker uploads an executable or web shell disguised as an image.	Uploading a <code>.php</code> file with a forged <code>Content-Type</code> header.
T-P5-02	T	Path traversal — attacker manipulates a filename to write outside the intended storage directory.	Filename containing <code>../../../../etc/cron.d/backdoor.</code>
T-P5-03	I	Unauthorised image download via IDOR or UUID guessing.	Guessing or enumerating image file UUIDs to retrieve another seller’s assets.
T-P5-04	I	Image metadata leaks original filenames or internal server paths.	API response returning a <code>file_path</code> field that exposes the server directory structure.

(continued)

ID	Cat.	Threat	Attack Vector
T-P5-05	T	Image file tampered with after upload, with integrity not verified on retrieval.	An attacker with filesystem access modifying product images.
T-P5-06	D	Storage exhaustion via large or numerous image uploads.	Automated upload of files near the size limit filling available disk space.

3.2.2.6 P-6: Invoice Generation Invoice generation is a sensitive process that consumes user-supplied data in a templating context and produces files on the server filesystem.

Table 13: Threats on Invoice Generation

ID	Cat.	Threat	Attack Vector
T-P6-01	T	Template injection via user-supplied data passed to a templating engine without sanitisation.	Product title or shipping address containing Jinja2 expressions such as <code>{{config}}</code> .
T-P6-02	I	Invoice files accessible without authorisation via direct path guessing.	Predictable or sequential file paths for generated invoices allowing enumeration.
T-P6-03	I	Invoice exposes sensitive data belonging to other buyers or internal identifiers.	Improper data aggregation or scoping during invoice generation.

3.2.2.7 P-7: Review Management The review system can be abused to distort seller reputation if purchase verification and rate controls are not enforced.

Table 14: Threats on Review Management

ID	Cat.	Threat	Attack Vector
T-P7-01	T	Fake review — buyer submits a review without having purchased the product.	Bypassing the purchase verification check via a direct API call.
T-P7-02	T	Review bombing — automated creation of negative reviews to damage a seller’s reputation.	Compromised accounts or race conditions in the review creation endpoint.
T-P7-03	T	Stored XSS injected in a review comment field.	Injecting script tags in review text that is later rendered on a frontend or in an export report.

3.2.2.8 P-8: Export Generation Export generation may aggregate sensitive sales data and consume significant resources, making it a target for information disclosure and denial-of-service.

Table 15: Threats on Export Generation

ID	Cat.	Threat	Attack Vector
T-P8-01	I	Export file contains data the requester is not authorised to access.	A seller’s export inadvertently including other sellers’ sales data due to missing scoping filters.
T-P8-02	D	Resource exhaustion through excessive or concurrent export generation requests.	Attacker triggering many simultaneous exports to consume CPU, memory, and disk resources.

3.2.2.9 P-9: Audit Logging The audit log is a critical security control. Threats against it directly undermine the platform’s ability to detect and investigate incidents.

Table 16: Threats on Audit Logging

ID	Cat.	Threat	Attack Vector
T-P9-01	T	Audit log tampering or deletion by a compromised admin account.	An admin with direct database access deleting or altering audit records.
T-P9-02	R	Insufficient logging — security-relevant events are not captured.	Missing entries for authorisation failures, order modifications, or file operations, preventing incident reconstruction.
T-P9-03	I	Sensitive data (passwords, tokens, addresses) inadvertently recorded in audit logs.	Logging full request bodies that include authentication payloads or personal information.

3.3 Data Flow Threats

In addition to process-level threats, threats were identified on the data flows that cross trust boundaries between the external network, the application server, and the data stores.

Table 17: Data Flow Threats

ID	Cat.	Threat	Data Flow	Attack Vector
T-DF-01	I	Man-in-the-middle interception of API traffic.	Client → API Server	Passive sniffing or active interception on an unencrypted HTTP connection.
T-DF-02	T	Tampering with request or response data in transit.	Client ↔ API Server	Active MITM attack modifying request payloads or injecting content into responses.
T-DF-03	I	Eavesdropping on unencrypted database connections.	API Server → PostgreSQL	Network-level interception of plaintext database traffic on the internal network.

(continued)

ID	Cat.	Threat	Data Flow	Attack Vector
T-DF-04	I	Unauthorised access to files stored on the server filesystem.	API Server → Filesystem	Misconfigured file permissions or a path traversal vulnerability allowing reads outside the intended storage root.

3.4 Risk Assessment Summary

The following table consolidates all identified threats with their DREAD scores and resulting priority levels. Scores are the average of Damage, Reproducibility, Exploitability, Affected Users, and Discoverability, each rated 1–3.

Table 18: DREAD Risk Scores per Threat

ID	Threat	D	R	E	A	D ₂	Score	Priority
T-P1-02	Credential stuffing	3	3	3	3	3	3.0	Critical
T-P4-01	Price manipulation	3	3	2	3	3	2.8	Critical
T-P5-06	Storage exhaustion	3	3	3	3	2	2.8	Critical
T-P5-02	Path traversal	3	2	2	3	2	2.4	High
T-P5-01	Malicious file upload	3	2	2	3	2	2.4	High
T-P6-01	Invoice template injection	3	2	2	2	2	2.2	High
T-P2-01	IDOR	3	2	2	2	2	2.2	High
T-P2-03	Privilege escalation	3	2	2	3	2	2.4	High
T-P1-03	JWT payload manipulation	3	2	2	3	2	2.4	High
T-DS-01	SQL injection	3	2	2	3	2	2.4	High
T-P3-02	Stored XSS (products)	2	2	2	3	2	2.2	High
T-P7-03	Stored XSS (reviews)	2	2	2	3	2	2.2	High
T-P4-05	Race condition (stock)	3	2	2	2	2	2.2	High
T-DF-01	MITM interception	3	2	2	3	2	2.4	High
T-P9-01	Audit log tampering	3	1	2	3	2	2.2	High
T-P9-02	Insufficient logging	2	2	2	3	2	2.2	High
T-P7-01	Fake reviews	2	2	2	2	2	2.0	High
T-P1-01	Brute force login	2	3	3	2	3	2.6	Critical
T-P1-05	Account lockout abuse	2	2	3	2	2	2.2	High
T-P5-04	Metadata leaks	1	2	2	2	2	1.8	Medium

(continued)

ID	Threat	D	R	E	A	D ₂	Score	Priority
T-DS-03	Database DoS (unbounded queries)	2	2	2	3	2	2.2	High
T-P9-03	Sensitive data in logs	2	2	1	3	2	2.0	High

3.5 Mitigation Traceability Matrix

Each identified threat is mapped to one or more planned mitigations and to the security requirement(s) that enforce those mitigations. This matrix provides end-to-end traceability from threat to control.

Table 19: Threats to Mitigations Mapping

Threat ID	Threat	Risk	Mitigation	Requirement(s)
T-P1-01	Brute force	Critical	Rate limiting, lockout	SR-AUTH-07, SR-AUTH-09
T-P1-02	Credential stuffing	Critical	Breached pwd check, lockout	SR-AUTH-04, SR-AUTH-07
T-P1-03	JWT manipulation	High	Strong signing, short TTL	SR-AUTH-05, SR-AUTH-06
T-P1-04	JWT secret leak	High	Env-var secrets only	SR-AUTH-06
T-P1-05	Lockout abuse	High	Progressive delay, IP limits	SR-AUTH-07, SR-AUTH-09
T-P1-06	Stale refresh token	High	Token invalidation on change	SR-AUTH-08
T-P2-01	IDOR	High	Resource-level checks, UUIDs	SR-AUTHZ-02, SR-AUTHZ-08
T-P2-02	Missing func. control	High	RBAC on every endpoint	SR-AUTHZ-01, SR-AUTHZ-02

(continued)

Threat ID	Threat	Risk	Mitigation	Requirement(s)
T-P2-03	Privilege escalation	High	Role field not user-editable	SR-AUTHZ-03
T-P2-04	Cross-seller tampering	High	Ownership check in service	SR-AUTHZ-05
T-P3-02	Stored XSS (products)	High	bleach strip, CSP	SR-INPUT-05, SR-INPUT-06, SR-COMM-05
T-P4-01	Price manipulation	Critical	Server-side snapshot	SR-DATA-06, SR-DATA-07
T-P4-05	Race cond. (stock)	High	DB-level locking	SR-DATA-10
T-P5-01	Malicious upload	High	MIME + magic byte validation	SR-INPUT-02
T-P5-02	Path traversal	High	UUID names, path check	SR-DATA-01, SR-INPUT-04
T-P5-06	Storage exhaustion	Critical	Size limit, quota	SR-INPUT-03, SR-DATA-09
T-P6-01	Template injection	High	Sandboxed Jinja2, escaping	SR-INPUT-05, SR-INPUT-10
T-P7-01	Fake reviews	High	Purchase verification	SR-BIZ-01
T-P7-02	Review bombing	High	One review per buyer (DB)	SR-BIZ-02
T-P7-03	Stored XSS (reviews)	High	bleach strip, CSP	SR-INPUT-06, SR-COMM-05
T-P9-01	Audit log tampering	High	INSERT/SELECT only DB user	SR-LOG-02
T-P9-02	Insufficient logging	High	Comprehensive event logging	SR-LOG-01, SR-LOG-03

(continued)

Threat ID	Threat	Risk	Mitigation	Requirement(s)
T-P9-03	Sensitive data in logs	High	Log field filtering	SR-LOG-04
T-DF-01	MITM interception	High	TLS 1.2+, HSTS	SR-COMM-01, SR-COMM-02
T-DS-01	SQL injection	High	ORM + parameterised queries	SR-INPUT-07
T-DS-03	Database DoS	High	Pagination, query timeouts	SR-INPUT-08

4 Architecture and Data Flows

This section presents the architecture design and data flow patterns that are the foundation of the SecureMarket platform. The design was based on the system requirements previously identified, with a focus on security, modularity, and maintainability.

4.1 Architecture Overview

The SecureMarket system adopts a layered architecture aligned with Domain-Driven Design (DDD) principles. This structure separates concerns across four primary layers: API, Application, Domain, and Infrastructure. According to the specification, the API layer handles HTTP requests, authentication, and validation, while the Application layer orchestrates use cases without containing business logic. The Domain layer encapsulates all business rules and aggregates, and the Infrastructure layer provides persistence, filesystem operations, and external integrations.

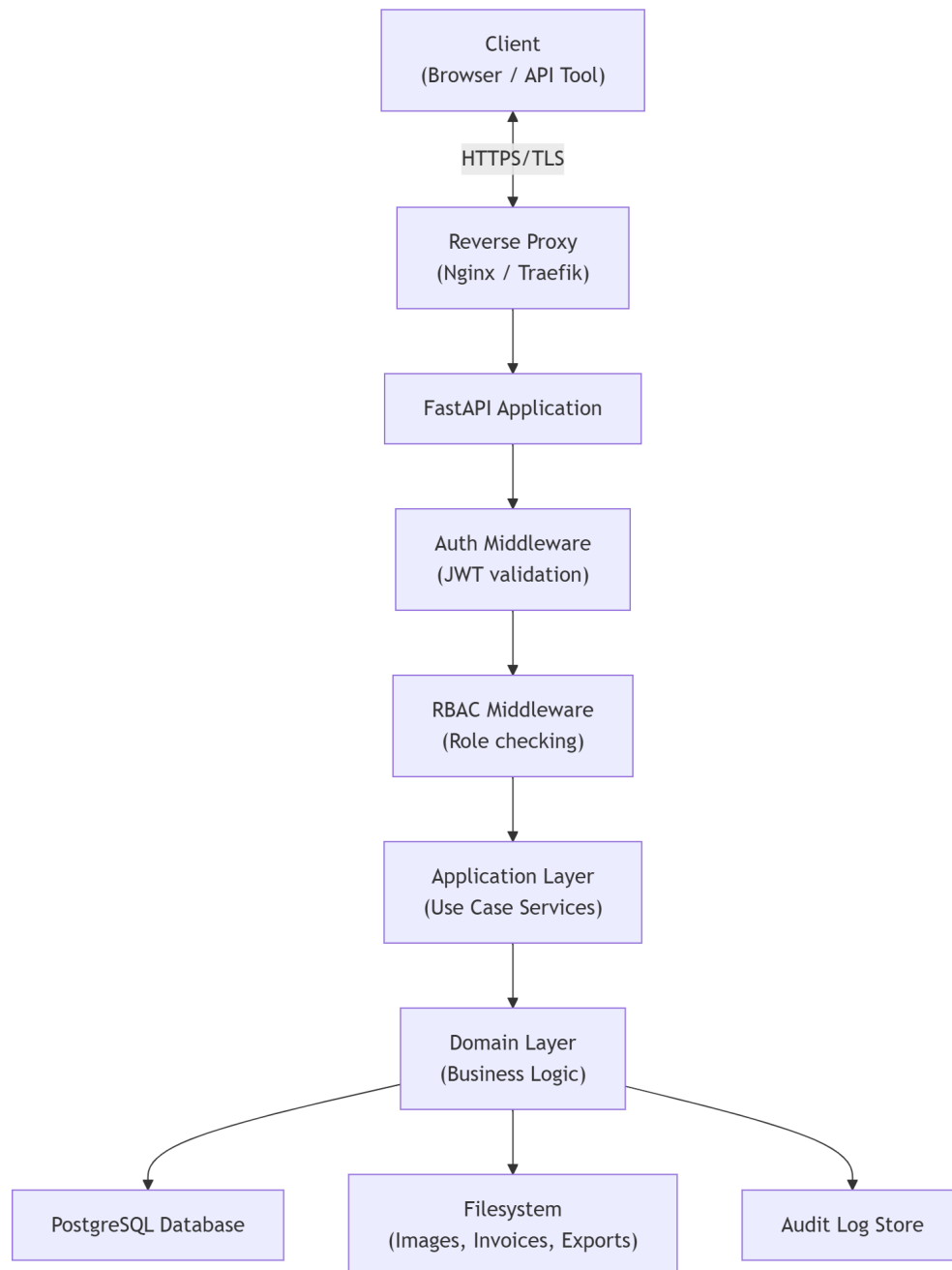


Figure 3: Architecture Overview Diagram

The system is deployed behind a reverse proxy (e.g., Nginx or Traefik), which enforces TLS and forwards requests to the FastAPI application. Internally, the application interacts with a PostgreSQL database for structured data and the server filesystem for product images, invoices, and export files. Audit logging is performed across all state-changing operations to ensure traceability.

The project structure reflects this separation of concerns, with dedicated modules for routing, services, domain models, repositories, and filesystem operations. This modularity supports maintainability and facilitates secure handling of sensitive operations such as image uploads and invoice generation.

4.2 Architecture Domain Model

The domain model is organized around three main aggregates: *User*, *Product*, and *Order*. Each aggregate enforces its own invariants and exposes controlled operations through its aggregate root.

4.2.0.1 User Aggregate The User aggregate models platform participants across three roles: ADMIN, SELLER, and BUYER. It includes value objects such as *EmailAddress* and *UserRole*. The documentation specifies that “email must be unique across all users” and that “deactivated users cannot authenticate” (“SecureMarket — Architecture & Domain Model”). Business rules include administrative control over user creation and role changes, password complexity requirements, and account lockout after repeated failed login attempts.

4.2.0.2 Product Aggregate The Product aggregate represents marketplace listings. It contains entities such as *ProductImage* and *Review*, and value objects including *Money*, *Category*, and *ProductStatus*. The system enforces constraints such as a maximum of ten images per product, valid MIME types for uploads, and the rule that “only ACTIVE products are visible to buyers.” Reviews are restricted to buyers who have purchased and received the product.

4.2.0.3 Order Aggregate The Order aggregate captures purchase transactions. It includes *OrderItem* entities and value objects such as *Address* and *OrderStatus*. Orders must contain at least one item, snapshot product prices at creation time, and follow strict status transitions. As noted in the documentation, “an invoice file is generated on the server filesystem when an order reaches CONFIRMED status.”

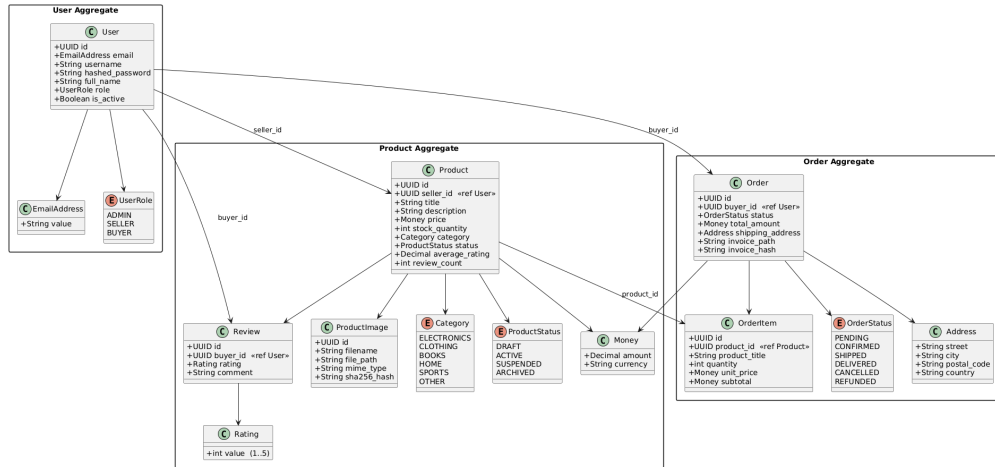


Figure 4: Domain Model Diagram

Cross-aggregate references are maintained through identifiers rather than direct object references, ensuring loose coupling and clear aggregate boundaries. Domain events (e.g., `ProductActivated`, `OrderPlaced`, `ReviewCreated`) support side effects such as audit logging, stock adjustments, and filesystem operations.

4.3 Data Flow Diagrams

4.3.1 DFD Level 0 – System Context

The data flow diagrams (DFDs) describe how information moves through the system across trust boundaries. At the highest level, external entities (Admin, Seller, Buyer) interact with the SecureMarket system through authenticated and authorized API requests. Data is stored in two primary locations: the PostgreSQL database and the server filesystem, as shown in Figure 5

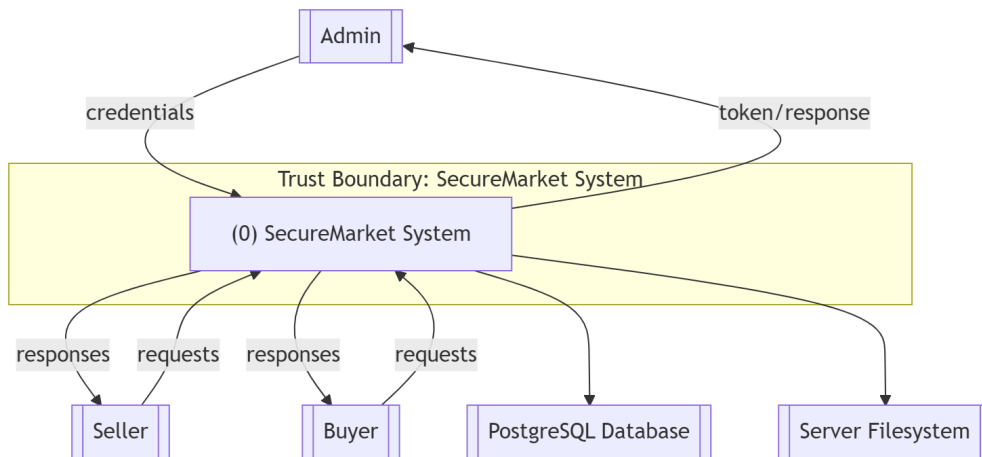


Figure 5: DFD Level 0

4.3.2 DFD Level 1 – Process Decomposition

The Level 1 DFD seen in figure 6 decomposes the system into major processes, including Authentication, Authorization, User Management, Product Management, Order Management, Image Management, Invoice Generation, Export Generation, and Audit Logging. Each process interacts with the appropriate data stores and enforces security controls. For example, image uploads involve MIME validation, SHA-256 hashing, and controlled filesystem writes, while order processing includes stock validation and price snapshotting.

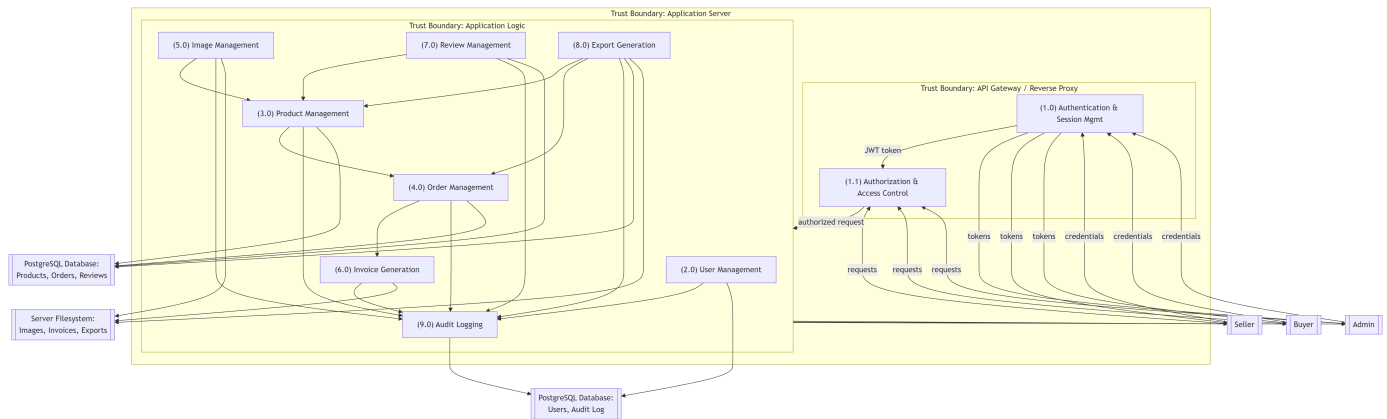


Figure 6: DFD Level 1

Lower-level diagrams further detail critical processes. Authentication includes credential validation, failed login tracking, and token issuance. Order processing validates product availability, snapshots prices, and triggers invoice generation. Image management enforces strict file validation and secure storage procedures. These flows collectively ensure that data is handled securely and consistently throughout the system.

4.3.3 DFD Level 2 – Authentication & Session Management

The Level 2 DFD for the Authentication module decomposes process 1.0 into six sub-processes, detailing the internal data flows that govern credential validation, token issuance, and session termination.

When a user submits their credentials, process (1.0.1) *Validate Credentials* checks the supplied email and password against the Users DB. On failure, the flow passes to process (1.0.2) *Record Failed Login Attempt*, which writes the failure to the Users DB and forwards the updated failure count to process (1.0.4) *Check Lockout Threshold*. If the threshold of five consecutive failures is reached, the account is locked in the Users DB, directly addressing threats T-P1-01 and T-P1-02. On a successful credential check, process (1.0.3) *Generate JWT Tokens* produces a short-lived access token and a longer-lived refresh token, which are returned to the user. When the user supplies a valid refresh token, process (1.0.5) *Refresh Token* validates it and triggers a new token pair via (1.0.3), mitigating T-P1-06 by ensuring revoked tokens are rejected. Finally, a logout request triggers process (1.0.6) *Invalidate Refresh Token*, which removes the token from the Users DB and writes a session termination event to the Audit Log. All failure and termination events are forwarded to the Audit Log, satisfying SR-AUTH-10.

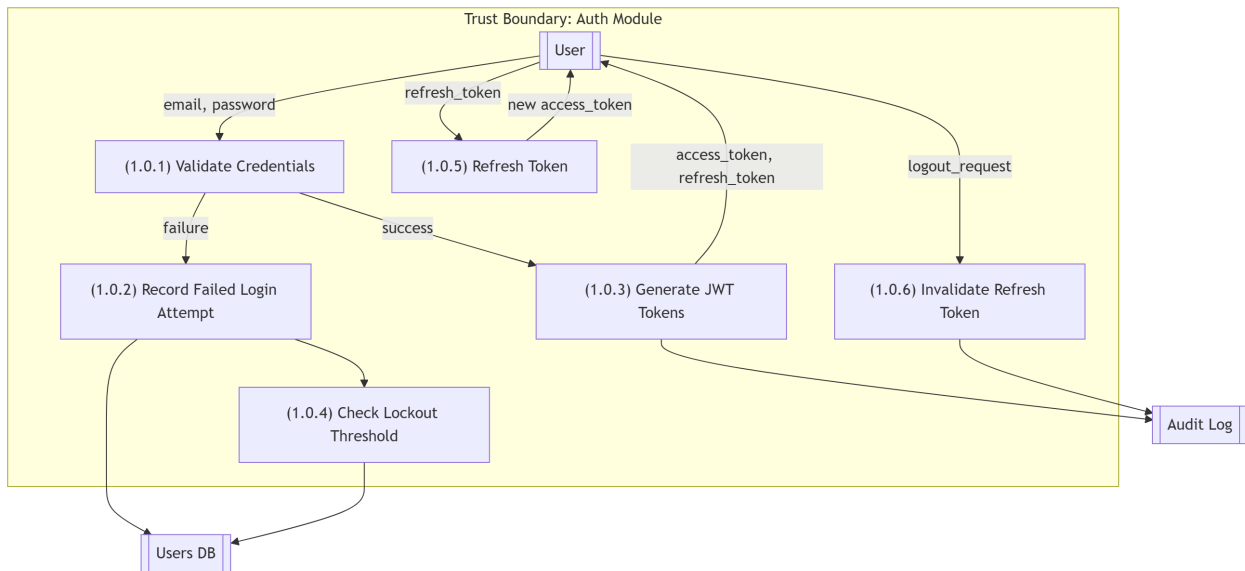


Figure 7: DFD Level 2 – Authentication & Session Management

4.3.4 DFD Level 2 – Order Management

The Level 2 DFD for Order Management decomposes process 4.0 into nine sub-processes covering the full order lifecycle: placement, confirmation, and cancellation. A buyer's order request enters process (4.0.1) *Validate Order Input*, which enforces schema and business rule checks. The validated request is forwarded to process (4.0.2) *Verify Product Availability & Pricing*, which queries the Products DB to confirm the product is in **ACTIVE** status and has

sufficient stock. If the product is unavailable, a rejection response is returned directly to the buyer. If available, process (4.0.3) *Snapshot Prices & Calculate Total* reads the current price from the Products DB server-side, ensuring the buyer cannot supply a manipulated price – directly mitigating T-P4-01 and satisfying SR-DATA-06 and SR-DATA-07. The snapshotted order is then persisted by process (4.0.4) *Create Order (PENDING)* to the Orders DB. On the seller side, process (4.0.5) *Confirm Order* handles the seller’s confirmation action. It triggers process (4.0.6) *Decrement Stock*, which applies a database-level lock on the Products DB row before decrementing, preventing the race condition described in T-P4-05 and satisfying SR-DATA-10. Confirmation also triggers process (4.0.7) *Trigger Invoice Generation*, which initiates the invoice sub-process described in the following section. When a buyer requests cancellation, process (4.0.8) *Cancel Order* validates that the order is in PENDING status before proceeding, rejecting invalid transitions per SR-BIZ-04. Cancellation calls process (4.0.9) *Restore Stock* to return the reserved quantity to the Products DB. All state-changing operations write events to the Audit Log, supporting non-repudiation against T-P4-04.

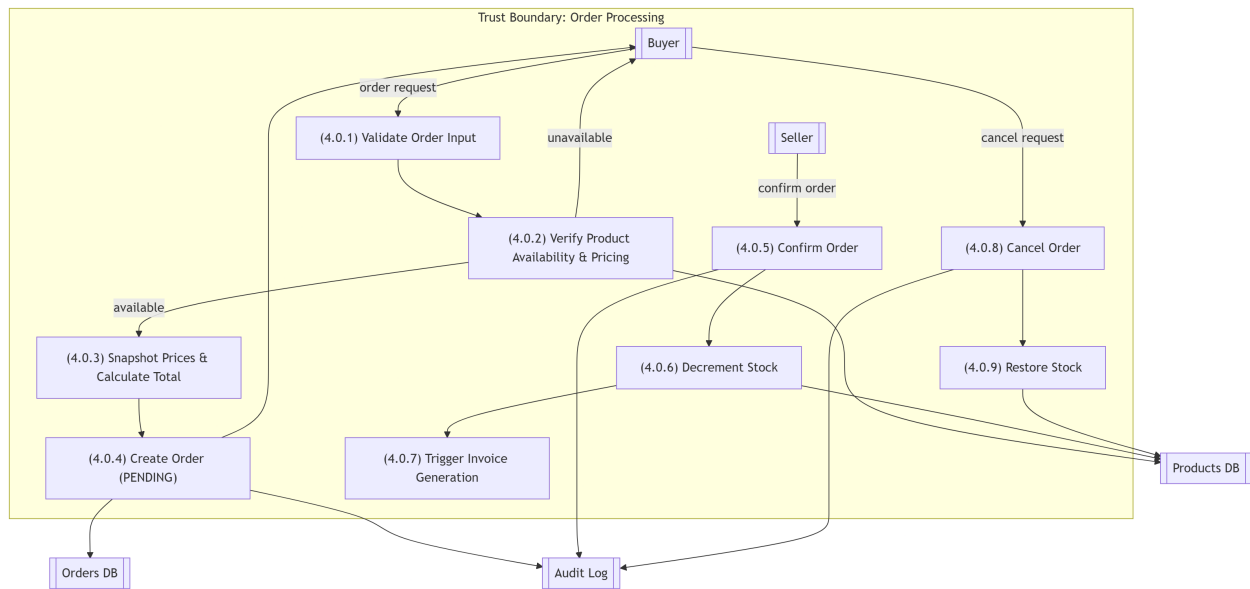


Figure 8: DFD Level 2 – Order Management

4.3.5 DFD Level 2 – Image Management

The Level 2 DFD for Image Management decomposes process 5.0 into eight sub-processes that govern secure file upload and storage. A seller submits a file and its metadata to process (5.0.1) *Validate Upload*, which checks both the file extension and the magic bytes of the file content against the allowed MIME types (`image/png`, `image/jpeg`, `image/webp`). Any file that fails either check is rejected and the seller is notified, mitigating T-P5-01 and satisfying SR-INPUT-02. Accepted files proceed to process (5.0.2) *Generate UUID Filename*, which replaces the original filename with a randomly generated UUID, eliminating user-controlled

path components and mitigating T-P5-02 in accordance with SR-DATA-01 and SR-INPUT-04. Process (5.0.3) *Compute SHA-256 Hash* then derives an integrity fingerprint of the file content, which will be verified on every subsequent download to detect tampering, addressing T-P5-05 per SR-DATA-02. Process (5.0.4) *Validate Path* performs a final check that the resolved absolute storage path falls within the configured storage root, providing a defence-in-depth guard against any residual path traversal vectors. The file is then written to the Filesystem Images Directory by process (5.0.5) *Write File to Disk*, after which process (5.0.6) *Set File Permissions (0640)* restricts access to owner read/write and group read, satisfying SR-DATA-04. Process (5.0.7) *Store Metadata* persists the UUID filename, original filename, file size, MIME type, and SHA-256 hash to the Images DB – never exposing the server path in any API response, mitigating T-P5-04. Finally, process (5.0.8) *Log Upload Event* writes a structured entry to the Audit Log, providing a complete record of every file operation per SR-LOG-03.

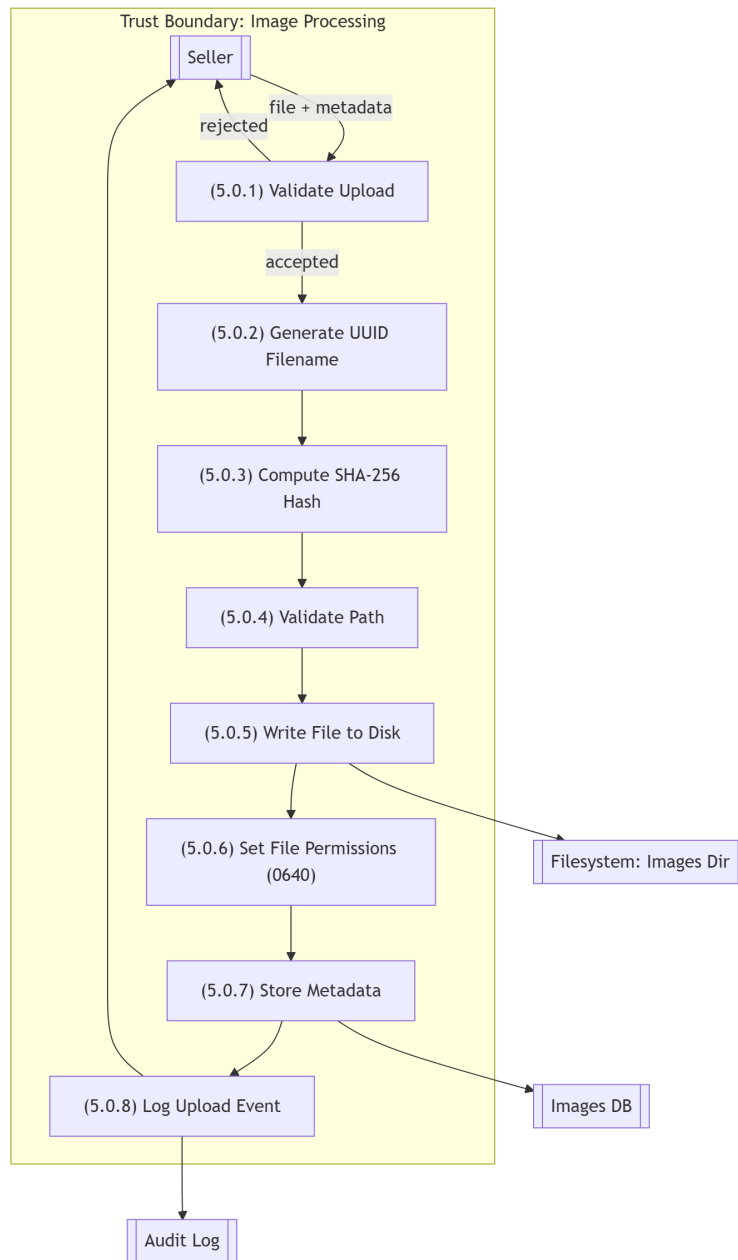


Figure 9: DFD Level 2 – Image Management

4.3.6 DFD Level 2 – Invoice Generation

The Level 2 DFD for Invoice Generation decomposes process 6.0 into six sub-processes triggered by an `order_confirmed` domain event, isolating invoice generation from the order placement flow. Upon receiving the event, process (6.0.1) *Retrieve Order Data* queries the Order Items DB, Products DB, and Users DB to assemble all data required for the invoice. This data is passed to process (6.0.2) *Compile Invoice Content*, which renders the invoice using a Jinja2

SandboxedEnvironment with auto-escaping enabled on all user-supplied fields – mitigating T-P6-01 and satisfying SR-INPUT-05. Before writing, process (6.0.3) *Ensure Buyer Invoice Directory Exists* verifies that the buyer’s dedicated directory is present within the storage root, preventing uncontrolled path creation. Process (6.0.4) *Write Invoice to Disk* writes the rendered file to the Filesystem Invoices Directory using a UUID-based filename, ensuring the invoice path is never predictable and mitigating T-P6-02. Process (6.0.5) *Compute SHA-256 of Invoice* derives an integrity hash of the generated file immediately after writing. Finally, process (6.0.6) *Update Order with invoice_path + hash* persists both the internal file path and the hash to the Orders DB, enabling integrity verification on every subsequent download per SR-DATA-02. A *Log Generation Event* is emitted to the Audit Log at the conclusion of the flow, recording the operation for traceability.

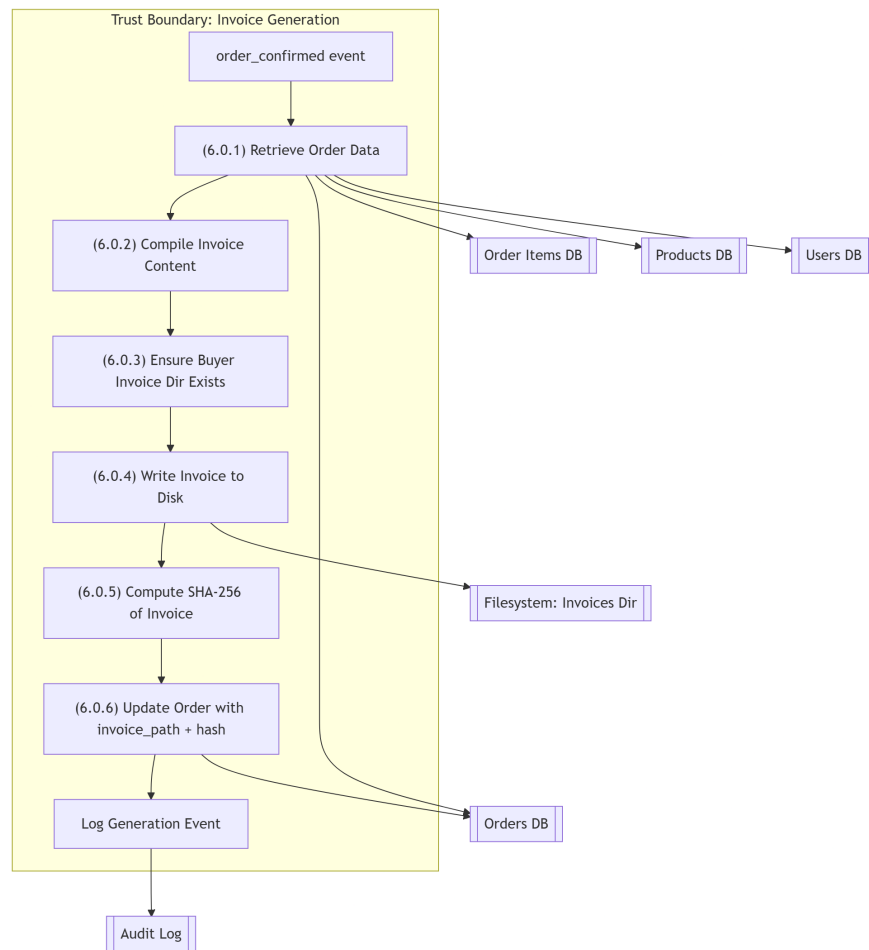


Figure 10: DFD Level 2 – Invoice Generation

5 Secure Development Requirements

In this section, we focus on detailing the security requirements for each category. Each requirement includes a unique identifier, a description, a priority level, the source that justifies it, and the related threats from the threat analysis document.

Requirements are derived from three sources:

- Threat model findings (threats identified via STRIDE/DREAD analysis);
- Industry best practices, OWASP ASVS v5.0 [5] and NIST SP 800-63B [6];
- Application business rules and project requirements.

5.1 Authentication & Session Management (SR-AUTH)

Authentication controls govern how users prove their identity and how sessions are maintained. Given that SecureMarket handles financial transactions and personal data, strong authentication is critical.

ID	Requirement	Priority	Source	Threats
SR-AUTH-01	The system shall authenticate users via email and password before granting access to any protected resource.	Critical	Best practice	T-EE-01
SR-AUTH-02	Passwords shall be encrypted when stored. Plaintext passwords shall never be stored or logged.	Critical	OWASP ASVS V2.4	T-P1-04
SR-AUTH-03	Passwords shall meet minimum complexity: at least 12 characters, including uppercase, lowercase, digits, and special characters.	High	NIST SP 800-63B	T-P1-01, T-P1-02
SR-AUTH-04	Passwords shall be checked against a list of known breached passwords.	High	NIST SP 800-63B	T-P1-02
SR-AUTH-05	The system shall issue short-lived JWT access tokens (max 15 minutes) and longer-lived refresh tokens (max 7 days).	High	Best practice	T-EE-02, T-P1-03

ID	Requirement	Priority	Source	Threats
SR-AUTH-06	JWT tokens shall be signed using a cryptographically strong algorithm (HS256 with 256-bit secret minimum, or RS256). The signing secret shall never be hardcoded.	Critical	OWASP ASVS V3.5	T-P1-03, T-P1-04
SR-AUTH-07	The system shall lock user accounts after 5 consecutive failed login attempts, with a 30-minute lockout duration.	High	Threat model	T-P1-01, T-P1-02
SR-AUTH-08	The system shall invalidate all active refresh tokens when a user changes their password or when an admin deactivates the account.	High	Best practice	T-P1-06, T-EE-02
SR-AUTH-09	The login endpoint shall be rate-limited to 5 attempts/min per IP and 20 attempts/min per account.	High	Threat model	T-P1-01, T-P1-05
SR-AUTH-10	All authentication events (success and failure) shall be logged with timestamp, user identifier, IP address, and result.	High	OWASP ASVS V7.1	T-P9-02

5.2 Authorization & Access Control (SR-AUTHZ)

Authorization controls ensure that authenticated users can only perform actions permitted by their role. SecureMarket implements Role-Based Access Control (RBAC) with three roles: ADMIN, SELLER, and BUYER.

ID	Requirement	Priority	Source	Threats
SR-AUTHZ-01	The system shall implement RBAC with three roles: ADMIN, SELLER, BUYER.	Critical	Project requirement	T-P2-02
SR-AUTHZ-02	Every API endpoint shall enforce both role-level and resource-level authorization.	Critical	Threat model	T-P2-01, T-P2-04
SR-AUTHZ-03	Users shall not be able to modify their own role. Role changes are restricted to ADMIN only.	Critical	Threat model	T-P2-03
SR-AUTHZ-04	Buyers shall only access their own orders. Sellers shall only see order items containing their products.	High	Business rule	T-P4-02
SR-AUTHZ-05	Sellers shall only modify products they own.	High	Business rule	T-P2-04
SR-AUTHZ-06	Buyers shall only view ACTIVE products. DRAFT/SUSPENDED/ARCHI products are visible only to the owning seller and admins.	High	Business rule	T-P3-03
SR-AUTHZ-07	All authorization failures shall be logged in the audit log.	High	OWASP ASVS V7.1	T-P9-02
SR-AUTHZ-08	The system shall use UUID v4 for all resource identifiers.	High	Best practice	T-P2-01

5.3 Data Security & Integrity (SR-DATA)

Data security requirements protect stored information from unauthorized access, tampering, and leakage. SecureMarket handles financial data (orders, prices) and user-generated files (images, invoices), both of which require strong integrity guarantees.

ID	Requirement	Priority	Source	Threats
SR-DATA-01	All image, invoice, and export files shall be stored with randomized UUID filenames. Original filenames stored in database only.	Critical	Threat model	T-P5-02, T-P5-04
SR-DATA-02	The system shall compute and store a SHA-256 hash of every uploaded image and generated invoice. Integrity shall be verified on download.	High	Threat model	T-P5-05
SR-DATA-03	Files shall be stored outside the application's web root and served only through authenticated API endpoints with Content-Disposition: attachment .	Critical	Threat model	T-P5-01, T-P6-02
SR-DATA-04	File permissions shall be set to 0640 (owner read/write, group read).	High	Best practice	T-DF-04
SR-DATA-05	The database shall use encryption at rest.	High	Best practice	
SR-DATA-06	Product prices in orders shall be snapshotted server-side at order creation time and shall be immutable thereafter.	Critical	Threat model	T-P4-01
SR-DATA-07	Order totals shall be calculated exclusively server-side. Client-supplied totals or prices shall never be trusted.	Critical	Threat model	T-P4-01
SR-DATA-08	Sensitive data (passwords, tokens, file contents, full addresses) shall never appear in application logs or API error responses.	Critical	OWASP ASVS V7.1	T-P9-03, T-P5-04

ID	Requirement	Priority	Source	Threats
SR-DATA-09	Per-seller image storage quotas shall be enforced (default: 200 MB).	Medium	Threat model	T-P5-06
SR-DATA-10	Stock updates during order confirmation shall use database-level locking to prevent race conditions.	High	Threat model	T-P4-05, T-P3-04

5.4 Communication Security (SR-COMM)

Communication security requirements ensure that data in transit between clients, the API server, and the database is protected from interception and tampering.

ID	Requirement	Priority	Source	Threats
SR-COMM-01	All client-to-server communication shall use HTTPS with TLS 1.2 or higher. HTTP shall redirect to HTTPS.	Critical	OWASP ASVS V9.1	T-DF-01
SR-COMM-02	The server shall set Strict-Transport-Security: max-age=31536000; includeSubDomains.	High	Best practice	T-DF-01
SR-COMM-03	Database connections shall use TLS (sslmode=require).	High	Best practice	T-DF-03
SR-COMM-04	CORS shall allow requests only from explicitly listed origins. No wildcard origins.	High	Best practice	
SR-COMM-05	The API shall set security headers: X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Content-Security-Policy: default-src 'none', Cache-Control: no-store for authenticated responses.	High	OWASP ASVS V14.4	T-P3-02

5.5 Input Validation & Data Handling (SR-INPUT)

Input validation requirements prevent malicious data from reaching application logic, the database, or the filesystem. This is especially critical given that SecureMarket processes file uploads, user-generated content, and financial values.

ID	Requirement	Priority	Source	Threats
SR-INPUT-01	All API inputs shall be validated using Pydantic schemas with strict type checking.	Critical	Best practice	T-DS-01, T-P3-02
SR-INPUT-02	Image uploads shall be validated by extension AND magic bytes. Allowed: <code>image/png</code> , <code>image/jpeg</code> , <code>image/webp</code> .	Critical	Threat model	T-P5-01
SR-INPUT-03	Maximum image upload size: 10 MB, enforced at API and filesystem layers.	High	Threat model	T-P5-06
SR-INPUT-04	All file storage paths shall be constructed server-side using UUID filenames. User-supplied filenames never used in path construction. Absolute path verified within storage root.	Critical	Threat model	T-P5-02
SR-INPUT-05	Text fields rendered in invoices/exports shall be sanitized for template injection. Template metacharacters escaped or stripped.	Critical	Threat model	T-P6-01
SR-INPUT-06	HTML tags shall be stripped from all user-supplied text fields using <code>bleach</code> lib.	High	Threat model	T-P3-02, T-P7-03
SR-INPUT-07	All database queries shall use SQLAlchemy ORM with parameterized queries. Raw SQL prohibited.	Critical	Best practice	T-DS-01
SR-INPUT-08	All list endpoints shall enforce pagination with max page size of 50.	Medium	Threat model	T-DS-03

ID	Requirement	Priority	Source	Threats
SR-INPUT-09	Product price must be validated as positive decimal (> 0 , max 2 decimal places). Stock must be non-negative integer.	High	Business rule	T-P4-01
SR-INPUT-10	Shipping address fields shall be validated for length and format. No template metacharacters allowed.	High	Threat model	T-P6-01

5.6 Third-Party Components (SR-3RD)

Third-party component requirements mitigate supply chain risks introduced by external libraries and container images used in the SecureMarket stack.

ID	Requirement	Priority	Source	Threats
SR-3RD-01	All Python dependencies shall be pinned to specific versions in <code>requirements.txt</code> .	High	Best practice	
SR-3RD-02	Dependencies shall be scanned using <code>pip-audit</code> and <code>Safety</code> in CI/CD. Builds fail on critical CVEs.	High	OWASP ASVS V14.2	
SR-3RD-03	A Software Bill of Materials (SBOM) shall be generated per release.	Medium	Best practice	
SR-3RD-04	Docker images shall use minimal base images (<code>python:3.12-slim</code>) and be scanned with Trivy.	High	Best practice	
SR-3RD-05	No development/debugging dependencies in production builds.	Medium	OWASP ASVS V14.2	

5.7 Logging & Monitoring (SR-LOG)

Logging requirements ensure that security-relevant events are recorded in a tamper-resistant manner, supporting incident response, compliance, and auditability.

ID	Requirement	Priority	Source	Threats
SR-LOG-01	The system shall maintain an immutable audit log recording all state-changing operations with: user ID, action, target resource, timestamp, IP address, result.	Critical	OWASP ASVS V7.1	T-P9-01, T-P9-02, T-EE-03
SR-LOG-02	The audit_log DB user shall have INSERT and SELECT only. No UPDATE/DELETE.	High	Threat model	T-P9-01
SR-LOG-03	Minimum logged events: all auth attempts, all CRUD on products/orders, all file operations, all status transitions, all authorization failures, all role changes, all review operations.	High	Threat model	T-P9-02
SR-LOG-04	Audit logs shall never contain passwords, tokens, or file contents.	Critical	OWASP ASVS V7.1	T-P9-03
SR-LOG-05	Error responses shall not expose stack traces, DB errors, or internal paths.	High	Best practice	T-P5-04
SR-LOG-06	Audit logs shall be retained for minimum 12 months.	Medium	Best practice	

5.8 Business Logic Security (SR-BIZ)

Business logic security requirements prevent abuse of application-specific workflows such as review manipulation, order tampering, and inventory exploitation.

ID	Requirement	Priority	Source	Threats
SR-BIZ-01	Reviews shall only be created by buyers who have a DELIVERED order for the product.	High	Threat model	T-P7-01
SR-BIZ-02	Each buyer may leave at most one review per product (enforced by DB unique constraint).	High	Threat model	T-P7-02
SR-BIZ-03	Order status transitions shall follow the defined state machine. Invalid transitions rejected.	High	Business rule	T-P4-03
SR-BIZ-04	Only PENDING orders can be cancelled by the buyer.	High	Business rule	T-P4-03
SR-BIZ-05	Only ACTIVE products with sufficient stock can be ordered.	High	Business rule	T-P4-05
SR-BIZ-06	Product status transitions shall follow: DRAFT→ACTIVE→SUSPENSE or DRAFT→ACTIVE→ARCHIVE	High	Business rule	
SR-BIZ-07	Maximum 10 images per product.	Medium	Business rule	T-P5-06

5.9 Requirements Traceability Matrix

The following matrix maps each requirement category to the threats it addresses and the corresponding OWASP ASVS v5.0 category.

Requirement Group	Threats Addressed	ASVS 5.0 Category
SR-AUTH-01 to SR-AUTH-10	T-EE-01, T-P1-01 to T-P1-06, T-EE-02, T-P9-02	V6 (Authentication), V7 (Session Management)
SR-AUTHZ-01 to SR-AUTHZ-08	T-P2-01 to T-P2-04, T-P3-03, T-P4-02, T-P9-02	V8 (Authorization)
SR-DATA-01 to SR-DATA-10	T-P4-01, T-P4-05, T-P5-02 to T-P5-06, T-P9-03	V11 (Cryptography), V14 (Data Protection)
SR-COMM-01 to SR-COMM-05	T-DF-01, T-DF-03, T-P3-02	V12 (Secure Communication)
SR-INPUT-01 to SR-INPUT-10	T-DS-01, T-P3-02, T-P5-01, T-P5-02, T-P5-06, T-P6-01, T-P7-03, T-DS-03, T-P4-01	V1 (Encoding and Sanitization), V2 (Validation and Business Logic), V5 (File Handling)
SR-3RD-01 to SR-3RD-05	Supply chain risks	V13 (Configuration), V15 (Secure Coding and Architecture)
SR-LOG-01 to SR-LOG-06	T-P9-01, T-P9-02, T-P9-03, T-EE-03	V16 (Security Logging and Error Handling)
SR-BIZ-01 to SR-BIZ-07	T-P7-01, T-P7-02, T-P4-03, T-P4-05, T-P5-06	V2 (Validation and Business Logic)

6 Secure Design

SecureMarket applies security-by-design principles: architectural decisions are made with attack resistance in mind, trust boundaries are explicitly defined, and every sensitive operation is subject to a corresponding control. This chapter documents the design patterns, principles, and specific decisions that reduce the system's attack surface and enforce its security invariants.

6.1 Secure Design Principles

The following core principles govern all design decisions in SecureMarket.

Table 29: Secure Design Principles

Principle	Application in SecureMarket
Least Privilege	Each user role (ADMIN, SELLER, BUYER) is granted only the permissions required for its function. The database application user holds the minimum SQL privileges needed to operate.
Defence in Depth	Security controls are layered: TLS at transport, JWT validation at the API gateway, RBAC at the route level, resource-level ownership checks at the service layer. No single failure exposes the system.
Fail Secure	Any unexpected condition (invalid token, malformed input, failed authorisation check) results in access denial and an appropriate HTTP error code (401/403/422). Exceptions are caught and return generic messages.
Separation of Concerns	The layered architecture (API → Application → Domain → Infrastructure) ensures business logic, authorisation, and infrastructure are decoupled. Security controls enforced in the API layer cannot be bypassed by calling the application layer directly.
Complete Mediation	Every API request passes through authentication and authorisation middleware before reaching business logic. Authorisation checks are enforced at both the route level and within service methods.
Economy of Mechanism	Security mechanisms are as simple as possible. Complex business rules (price integrity, stock locking) are enforced at a single, testable point in the domain layer, not scattered across multiple layers where they could be independently broken.

6.2 Role-Based Access Control Design

SecureMarket implements RBAC with three roles and a permissions matrix enforced at every endpoint. Role changes are restricted to ADMIN only and cannot be self-applied.

Table 30: Role-Based Access Control Permissions Matrix

Operation	ADMIN	SELLER	BUYER
Register	×	✓	✓
CRUD products & images	✓ (any)	✓ (own)	×
Browse active products	✓	✓	✓
Place / cancel orders	✓	×	✓ (own)
Fulfil orders (status update)	✓ (any)	✓ (own products)	×
Download invoice	✓	✓ (own)	✓ (own)
Leave reviews	×	×	✓ (delivered)
View audit log	✓	×	×
Change user roles	✓	×	×

7 Secure Architecture

This chapter documents SecureMarket’s secure architecture, describing how the system is structured to enforce security properties across all layers. Architecture decisions are directly traceable to threats identified in the threat analysis and requirements defined in the security requirements document.

7.1 Layered Architecture Overview

SecureMarket follows a four-layer architecture aligned with Domain-Driven Design. Each layer has a defined security responsibility and may only communicate with adjacent layers, preventing security control bypass.

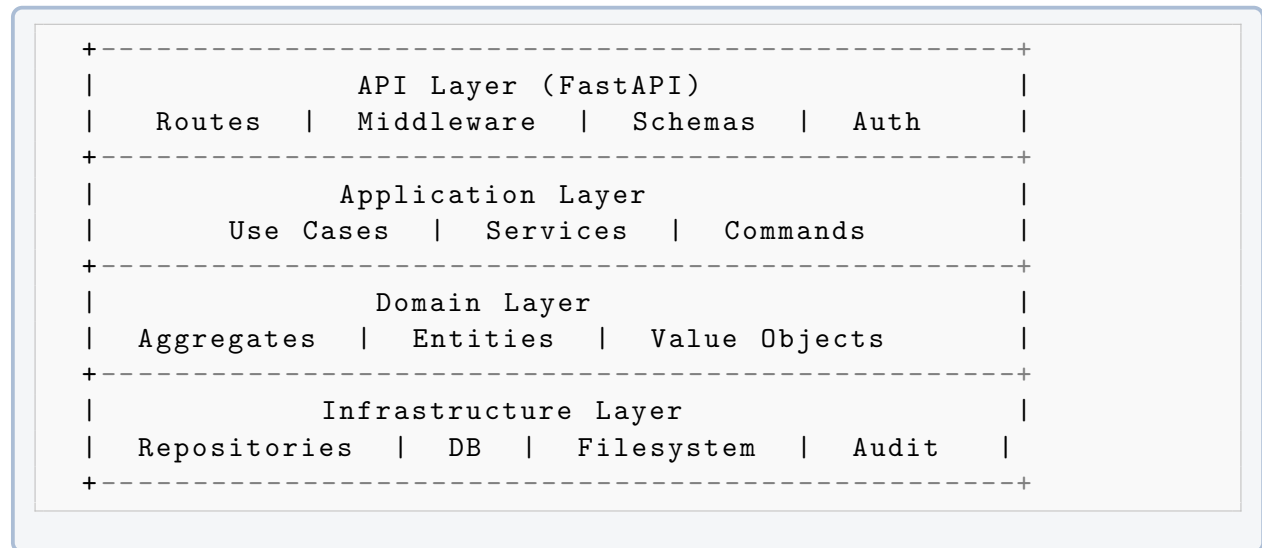


Figure 11: SecureMarket Four-Layer Architecture

Table 31: Layer Responsibilities

Layer	Components	Security Responsibilities
API Layer	FastAPI routes, Pydantic schemas, middleware	JWT validation, RBAC enforcement, rate limiting, security header injection, input schema validation, error normalisation.
Application Layer	Use case services, command handlers	Resource-level ownership checks, business rule enforcement, audit log emission, server-side price and total calculation.

Layer	Components	Security Responsibilities
Domain Layer	Aggregates, entities, value objects, domain events	Enforcement (price positivity, stock non-negativity, valid state transitions), domain-level validation, purchase eligibility for reviews.
Infrastructure Layer	SQLAlchemy repositories, filesystem service, audit store	Parameterised ORM queries, UUID file naming, path validation, database-level stock locking, TLS-encrypted DB connections, append-only audit writes.

7.2 Deployment Architecture

SecureMarket is deployed as a containerised stack using Docker Compose, with a reverse proxy handling TLS termination. No component is directly exposed to the internet without passing through the proxy.

Table 32: Deployment Components and Security Configuration

Component	Technology	Security Configuration
Reverse Proxy	Nginx	TLS 1.2+ termination. HTTP → HTTPS redirect (301). HSTS header (<code>max-age=31536000; includeSubDomains</code>).
API Application	FastAPI (Python)	Runs as non-root user in container. Security headers middleware. Secrets via environment variables only.
Database	PostgreSQL(latest stable version)	TLS connections. Encryption at rest. Minimum-privilege DB users. Not exposed outside the Docker network.
File Storage	Docker volume	Stored outside web root. Not mounted read-write outside the application container. UUID-named files only.
Container Images	<code>python:3.12-slim</code>	Minimal base image. No development dependencies. No debug tools. Non-root user.
Secrets Management	Environment variables	JWT signing secret, DB credentials never hardcoded. <code>.env</code> excluded from version control

7.3 Security Middleware Stack

Every inbound HTTP request passes through a sequential middleware stack before reaching any business logic. The order is architecturally significant (earlier layers reject invalid requests before later, more expensive layers execute).

Table 33: Security Middleware Stack

#	Middleware	Function
1	TLS Termination (Proxy)	Ensures all traffic is encrypted. Rejects plaintext HTTP with 301 redirect. Sets HSTS on all responses.
2	Rate Limiting	Enforces per-IP and per-account request limits. Login endpoint: 5/min per IP. Returns HTTP 429 on excess.
3	Security Headers	Injects on all responses: <code>Strict-Transport-Security,</code> <code>X-Content-Type-Options: nosniff,</code> <code>X-Frame-Options: DENY,</code> <code>Content-Security-Policy: default-src 'none',</code> <code>Cache-Control: no-store.</code>
4	JWT Authentication	Validates JWT signature (HS256/RS256) and expiry claim. Rejects expired, malformed, or revoked tokens with HTTP 401.
5	RBAC Authorisation	Checks role permissions for the requested endpoint. Returns HTTP 403 for insufficient role.
6	Input Validation (Pydantic)	Validates request body, query, and path parameters against Pydantic schemas. Returns HTTP 422 on validation failure.
7	Resource Authorisation (Service)	Service-level check: verifies the requesting user owns or has rights to the specific resource (e.g. seller owns the product). Returns HTTP 403 on failure.

7.4 Authentication Architecture

The authentication subsystem implements a dual-token JWT flow with server-side refresh token management.

Table 34: Authentication Architecture

Token Type	Properties	Security Controls
Access Token	JWT, Expiry: 15 minutes. Claims: <code>user_id</code> , <code>role</code> , <code>jti</code> .	Short expiry limits damage window. Stateless validation at every request. Signing secret from environment variable only.
Refresh Token	Opaque or JWT. Expiry: 7 days. Hash stored in database.	Server-side deny-list. Invalidated on: password change, logout, admin deactivation. Single-use: each refresh issues a new refresh token.
Password Storage	bcrypt	Never stored plaintext. Never logged. Minimum complexity enforced (SR-AUTH-03).
Account Lockout	5 consecutive failures trigger 30-minute lockout.	Applied per-account and per-IP. Prevents brute force and credential stuffing (T-P1-01, T-P1-02). Lockout events logged to audit log.

7.5 Database Security Architecture

Table 35: Database Security Architecture

Control	Implementation Detail
Encrypted Transport	All connections use TLS. Plaintext PostgreSQL connections are rejected at the server level.
Encrypted at Rest	Database volume encryption enabled. Protects against data exposure if container storage is accessed directly.
Least-Privilege DB Users	<code>app_user</code> : SELECT, INSERT, UPDATE, DELETE on application tables only. <code>audit_user</code> : INSERT, SELECT on <code>audit_log</code> only. No DDL privileges granted to either.
ORM-Only Queries	SQLAlchemy ORM used exclusively
UUID Primary Keys	All resources use UUID v4 primary keys. Prevents enumeration attacks (T-P2-01) and provides unpredictable, non-sequential identifiers.

Control	Implementation Detail
Connection Pooling	SQLAlchemy connection pool with <code>max_overflow</code> limit prevents DB connection exhaustion (T-DS-03). <code>statement_timeout</code> configured at the server level.
Row-Level Locking	<code>SELECT ...FOR UPDATE</code> used during stock decrement. Prevents negative stock from concurrent order confirmations (T-P4-05).

7.6 CI/CD Pipeline Security Architecture

Security controls are automated in the GitHub Actions CI/CD pipeline. Deployments are blocked if any security gate fails. The pipeline sequence is: commit → SAST → SCA → unit/integration tests → container build → container scan → DAST → deploy.

Table 36: CI/CD Pipeline Security Stages and Verification Controls

Stage	Tool(s)	What It Checks	Blocks Deploy?
SAST	Bandit, Semgrep	Insecure patterns, raw SQL, <code>eval()</code> , <code>os.system()</code> , missing validation.	YES
SCA	pip-audit, Safety	Known CVEs in Python dependencies.	YES
Unit & Integration Tests	pytest, pytest-cov, httpx	All tests pass.	YES
Container Scan	Trivy	CVEs in Docker base image and OS packages.	YES
DAST	OWASP ZAP	Runtime injection, auth bypass, misconfigurations.	YES (High+)
SBOM Generation	CycloneDX / SPDX	Software Bill of Materials generated per release.	No

8 Security Testing Plan

The security testing plan defines the methodology, tools, test cases, and traceability matrix for validating the security requirements identified in Phase 1. Testing follows a multi-layered approach integrated into the SSDLC pipeline, covering static analysis, software composition analysis, dynamic analysis, and manual abuse case testing.

8.1 Testing Methodology

Table 37: Security Testing Methodology

Category	Timing	Tools	Purpose
SAST	Every commit / PR	Bandit, Semgrep	Detect insecure code patterns, hardcoded secrets, dangerous functions, raw SQL.
SCA	Every build	pip-audit, Safety, Trivy	Detect known CVEs in dependencies and container images.
DAST	Per sprint (CI)	OWASP ZAP (baseline + API scan)	Detect runtime vulnerabilities: injection, auth flaws, misconfigurations.
Manual Testing	Per sprint	pytest scripts, manual review	Test abuse cases, business logic flaws, authorisation bypasses, race conditions.

8.2 Quality Gates

All gates must pass before any deployment to staging or production. Failures block the pipeline and must be remediated before the build can proceed.

Table 38: Security Quality Gates

Gate	Pass Criteria	Blocks Deploy?
SAST (Bandit + Semgrep)	Zero high or critical findings	YES
SCA (pip-audit + Safety)	Zero critical CVEs in dependencies	YES

Gate	Pass Criteria	Blocks Deploy?
Unit Tests	All tests pass; coverage $\geq 80\%$	YES
Integration Tests	All API integration tests pass	YES
Security Tests	All auth/authz test cases pass	YES
Container Scan (Trivy)	Zero critical CVEs in Docker image	YES
DAST (OWASP ZAP)	Zero high-risk ZAP alerts after active scan	YES

8.3 Static Analysis (SAST) Test Cases

Bandit performs Python security linting. **Semgrep** applies rule packs: `p/owasp-top-ten`, `p/python`, `p/jwt`, `p/sql-injection`.

Table 39: Static Analysis Security Test Cases

ID	Test	Expected Result	Related Threats
SAST-01	Scan for hardcoded secrets (JWT secrets, DB passwords, API keys) in source code and config files.	Zero findings	T-P1-04
SAST-02	Scan for raw SQL or string-formatted queries (f-strings in queries, string concatenation).	Zero findings	T-DS-01
SAST-03	Scan for unsafe OS calls: <code>os.system()</code> , <code>subprocess</code> with <code>shell=True</code> , unvalidated path construction.	Zero findings	T-P5-02
SAST-04	Scan for dangerous Python functions: <code>eval()</code> , <code>exec()</code> , <code>pickle.loads()</code> , <code>yaml.load()</code> without Loader.	Zero findings	T-P6-01

ID	Test	Expected Result	Related Threats
SAST-05	Verify all API endpoints have input validation via Pydantic schemas (no bare <code>dict</code> or <code>Any</code> types on routes).	All endpoints covered	T-P3-02, T-DS-01
SAST-06	Verify all Jinja2 template environments use <code>SandboxedEnvironment</code> (not standard <code>Environment</code>).	All environments sandboxed	T-P6-01

8.4 Software Composition Analysis (SCA) Test Cases

Table 40: Software Composition Analysis Test Cases

ID	Test	Expected Result	Requirement
SCA-01	Scan <code>requirements.txt</code> for known CVEs using pip-audit and Safety.	Zero critical/high CVEs	SR-3RD-02
SCA-02	Verify all Python dependencies are pinned to specific versions (no ranges or wildcards).	All pinned	SR-3RD-01
SCA-03	Scan Docker base image (<code>python:3.12-slim</code>) with Trivy for OS-level vulnerabilities.	Zero critical CVEs	SR-3RD-04
SCA-04	Verify no development or debugging dependencies present in the production Docker image.	No dev packages	SR-3RD-05
SCA-05	Generate and validate Software Bill of Materials (SBOM) in CycloneDX or SPDX format per release.	Valid SBOM generated	SR-3RD-03

8.5 Dynamic Analysis (DAST) Test Cases

Two scan modes are employed. **Baseline Scan** (passive, every build): misconfigurations, missing headers, information disclosure. **API Scan** (active, per sprint): uses the OpenAPI specification from `/openapi.json` against the staging environment.

Table 41: Dynamic Analysis Security Test Cases

ID	Test	Method	Expected Result	Related Threats
DAST-01	SQL injection on all query, path, and body parameters.	ZAP Active	Zero SQL injection alerts	T-DS-01
DAST-02	Reflected and stored XSS on all text input fields.	ZAP Active	Zero XSS alerts	T-P3-02, T-P7-03
DAST-03	Verify all required security headers present: HSTS, X-Content-Type-Options, X-Frame-Options, CSP, Cache-Control.	ZAP Baseline	All headers present and correctly configured	SR-COMM-05
DAST-04	Path traversal on all file-related endpoints.	ZAP + Manual	Zero path traversal vulnerabilities	T-P5-02
DAST-05	Access all protected endpoints without valid token or with a tampered token.	ZAP Active	All return HTTP 401 or 403	T-P2-02
DAST-06	TLS configuration: TLS 1.2+, valid certificate chain.	testssl.sh	TLS 1.2+, no weak ciphers	T-DF-01
DAST-07	Trigger 404, 422, 500 errors and verify no stack traces, internal paths, or DB errors exposed.	ZAP Baseline	Generic error messages only	SR-LOG-05

8.6 Manual Security Testing

Manual tests directly correspond to the abuse cases documented in the threat analysis. Each test validates one or more security requirements and verifies a specific threat is mitigated.

8.6.1 Authentication Abuse Tests

Table 42: Authentication Abuse Test Cases

ID	Abuse Case	Test Steps & Expected Result	Requirements
MST-01	AC-01: Credential Stuffing / Brute Force	Send 10 login attempts with wrong passwords for the same account. Expected: Account locked after 5 attempts. HTTP 429/403 during lockout. Unlocks after 30 minutes.	SR-AUTH-07, SR-AUTH-09
MST-02	IP-Based Rate Limiting	Send 5 login requests from the same IP within 1 minute. Expected: Returns HTTP 429 after rate limit exceeded.	SR-AUTH-09
MST-03	JWT Token Tampering	Decode valid JWT. Modify <code>role</code> claim to <code>ADMIN</code> . Re-encode without a valid signature. Submit request. Expected: Returns HTTP 401 (invalid signature). Access denied.	SR-AUTH-06
MST-04	Token Refresh After Deactivation	Authenticate user. Admin deactivates account. Attempt token refresh with still-valid refresh token. Expected: Returns HTTP 401. All refresh tokens invalidated.	SR-AUTH-08

8.6.2 Authorisation Abuse Tests

Table 43: Authorisation Abuse Test Cases

ID	Abuse Case	Test Steps & Expected Result	Requirements
MST-05	AC-04: IDOR on Orders	Authenticate as Buyer A. Request GET <code>/orders/{buyer_b_order_id}</code> . Expected: Returns HTTP 403. Buyer A cannot access Buyer B's orders.	SR-AUTHZ-02, SR-AUTHZ-04
MST-06	IDOR on Products (Seller)	Authenticate as Seller A. Try PUT <code>/products/{seller_b_product_id}</code> . Expected: Returns HTTP 403. Sellers cannot modify other sellers' products.	SR-AUTHZ-05
MST-07	Role Parameter Tampering	Authenticate as Buyer. Send PUT <code>/users/{own_id}</code> with <code>{"role": "ADMIN"}</code> . Expected: Role unchanged. Field ignored or HTTP 403 returned.	SR-AUTHZ-03
MST-08	Function-Level Access Control	Authenticate as Buyer. Request GET <code>/api/v1/users</code> and GET <code>/api/v1/audit-log</code> . Expected: Both return HTTP 403.	SR-AUTHZ-01, SR-AUTHZ-02
MST-09	Buyer Accesses Draft Product	Seller creates a DRAFT product. Buyer requests GET <code>/products/{draft_id}</code> . Expected: Returns HTTP 404 or 403.	SR-AUTHZ-06

8.6.3 Business Logic Abuse Tests

Table 44: Business Logic Abuse Test Cases

ID	Abuse Case	Test Steps & Expected Result	Requirements
MST-11	AC-02: Price Manipulation	Place order with request body containing "unit_price": 0.01. Expected: Order total matches server-side product price. Client-supplied value ignored.	SR-DATA-06, SR-DATA-07
MST-12	Unavailable product order	Product has stock=1. Simultaneously submit 2 order confirmations via concurrent threads. Expected: Only 1 succeeds. Stock never goes negative. Second returns insufficient stock error.	SR-DATA-10
MST-13	AC-04: Fake Review (No Purchase)	Authenticate as Buyer with no delivered order for a product. Try POST /products/{pid}/reviews. Expected: Returns HTTP 403 (no delivered order for this product).	SR-BIZ-01
MST-14	Duplicate Review	Buyer with a delivered order submits a second review for the same product. Expected: Returns HTTP 409 (conflict) or 400. DB unique constraint enforced.	SR-BIZ-02
MST-15	Invalid Order Status Transition	Attempt to move an order from PENDING directly to DELIVERED (skipping CONFIRMED and SHIPPED). Expected: Returns HTTP 400 (invalid transition per state machine).	SR-BIZ-03
MST-16	Cancel Non-Pending Order	Order is in CONFIRMED status. Buyer attempts to cancel. Expected: Returns HTTP 400 (only PENDING orders can be cancelled by buyers).	SR-BIZ-04

8.6.4 File Operation Abuse Tests

Table 45: File Operation Abuse Test Cases

ID	Abuse Case	Test Steps & Expected Result	Requirements
MST-17	AC-05: Path Traversal Upload	Upload image with filename <code>../../../../etc/passwd</code> , with null bytes, and as a symlink target. Expected: File stored with UUID name. No files written outside storage root. Returns HTTP 422 for malicious names.	SR-INPUT-04, SR-DATA-01
MST-18	AC-03: Malicious File Upload	Upload <code>.php</code> file renamed to <code>.png</code> . Upload valid PNG extension with embedded executable content. Expected: Upload rejected on magic byte mismatch. Returns HTTP 422.	SR-INPUT-02
MST-19	AC-09: Storage Exhaustion	Upload 10 images to one product (maximum). Attempt an 11th upload. Expected: Returns HTTP 400 (maximum images per product exceeded).	SR-DATA-09, SR-BIZ-07
MST-20	Direct File Path Access	Identify image UUID. Attempt to access the file directly via HTTP outside authenticated API endpoints. Expected: File not accessible. Returns HTTP 404 from reverse proxy. Files not in web root.	SR-DATA-03

8.7 Test Traceability Matrix

The following matrix links each test case to the security requirements and threats it verifies, ensuring complete coverage from threat identification through to test validation.

Table 46: Security Test Traceability Matrix

Test ID	Requirement(s)	Threat(s)	Abuse Case
SAST-01	SR-AUTH-06	T-P1-04 (JWT secret exposure)	—
SAST-02	SR-INPUT-07	T-DS-01 (SQL injection)	—
SAST-06	SR-INPUT-05	T-P6-01 (SSTI)	AC-06
SCA-01	SR-3RD-02	Supply chain CVEs	—
DAST-01	SR-INPUT-07	T-DS-01 (SQL injection)	—
DAST-02	SR-INPUT-06	T-P3-02, T-P7-03 (XSS)	—
DAST-04	SR-INPUT-04	T-P5-02 (path traversal)	AC-05
MST-01	SR-AUTH-07, SR-AUTH-09	T-P1-01, T-P1-02 (brute force, stuffing)	AC-01
MST-03	SR-AUTH-06	T-P1-03 (JWT tampering)	—
MST-05	SR-AUTHZ-02, SR-AUTHZ-04	T-P2-01 (IDOR)	AC-04
MST-07	SR-AUTHZ-03	T-P2-03 (privilege escalation)	—
MST-11	SR-DATA-06, SR-DATA-07	T-P4-01 (price manipulation)	AC-02
MST-12	SR-DATA-10	T-P4-05 (stock race condition)	AC-08
MST-13	SR-BIZ-01	T-P7-01 (fake review)	AC-07
MST-17	SR-INPUT-04, SR-DATA-01	T-P5-02 (path traversal)	AC-05
MST-18	SR-INPUT-02	T-P5-01 (malicious file upload)	AC-03

Bibliography

- [1] OWASP Foundation. API2:2023 Broken Authentication. <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/>, 2023. Accessed: April 2026.
- [2] OWASP Foundation. API4:2023 Unrestricted Resource Consumption. <https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/>, 2023. Accessed: April 2026.
- [3] Adam Shostack. Threat Modeling: Designing for Security, 2014. Original source of the STRIDE threat modelling methodology.
- [4] OWASP Foundation. Threat Modeling — OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html, 2023. Accessed: April 2026.
- [5] OWASP Foundation. Application Security Verification Standard (ASVS) 5.0.0. <https://owasp.org/www-project-application-security-verification-standard/>, 2025. Accessed: April 2026.
- [6] National Institute of Standards and Technology. NIST Special Publication 800-63B: Digital Identity Guidelines — Authentication and Lifecycle Management. <https://pages.nist.gov/800-63-3/sp800-63b.html>, 2017. Accessed: April 2026.